COVOF 2D — Guide de Développement et Architecture Logicielle

SUBCOVOF 2D — Guide de Développement et Architecture Logicielle

Équipe de Développement & Recherche COVOF 2D Direction Scientifique & Numérique

12 novembre 2026

Table des matières

1 Architecture Logicielle Hybride C++ / Fortran **4**
1.1 Vision d'Ensemble et Paradigmes de Conception 4
1.1.1 Les Paradigmes en Présence 4
1.2 Cartographie Intégrale de l'Arborescence du Dépôt 5
1.2.1 1. Fichiers de la Branche C++ (`COVOF_2D-C++`) 5
1.2.2 2. Fichiers de la Branche Fortran (`COVOF_2D-FORTRAN`) 7
1.3 Graphe de Dépendances et Cycle d'Exécution Complet 8
1.3.1 Graphe de Dépendances Modulaire 8
1.3.2 Cycle d'Exécution d'un Pas de Temps 9

2 Interopérabilité C++ ↔ Fortran **11**
2.1 2.1. Standards et Conventions d'Interface : `iso_c_binding` et `extern "C"` 11
2.1.1 2.1.1. Directive `extern "C"` côté C++17 11
2.1.2 2.1.2. Module Intrinsèque `iso_c_binding` côté Fortran 2018 12
2.2 2.2. Table Complète de Correspondance des Types et Structures 12
2.2.1 2.2.1. Types Scalaires Primitifs 12
2.2.2 2.2.2. Structures Dérivées et Alignement Mémoire (`bind(C)`) 13
2.2.2.1 Définition C++17 13
2.2.2.2 Définition Fortran 2018 Correspondante 13
2.2.3 2.2.3. Conversion de Pointeurs Opaques : `c_f_pointer` et `c_loc` 14
2.3 2.3. Gestion de la Disposition Mémoire (Row-Major vs Column-Major) 14
2.3.1 2.3.1. Théorie de l'Adressage Linéaire 14
2.3.2 2.3.2. Règle d'Or de Parité Mémoire dans COVOF 2D 15
2.4 2.4. Gestion des Chaînes de Caractères, Entrées/Sorties et Erreurs 15
2.4.1 2.4.1. Passage des Chaînes de Caractères 15
2.4.2 2.4.2. Synchronisation des Unités Logiques d'E/S 16
2.4.3 2.4.3. Propagation Sécurisée des Codes d'Erreur 16
2.5 2.5. Catalogue des Points d'Entrée et Passerelles HPC 17
2.5.1 2.5.1. Traduction des Communicateurs MPI (`MPI_Comm_c2f` / `MPI_Comm_f2c`) . 17
2.5.2 2.5.2. Tableau Récapitulatif des Fonctions Jumelles C++ / Fortran 17

3 Structures de Données, Mémoire et Parallélisme HPC **19**
3.1 3.1. Représentation Interne du Maillage et des Champs 19
3.1.1 3.1.1. Topologie de Grille MAC Décalée (*Marker-And-Cell*) 19
3.1.2 3.1.2. Choix de Conception : Structure de Tableaux (SoA) vs Tableau de Structures (AoS) 20
3.1.3 3.1.3. Alignement Mémoire et Vectorisation SIMD (AVX2 / AVX-512) 20
3.2 3.2. Gestion des Cellules Fantômes (*Halo / Ghost Cells*) 20
3.2.1 3.2.1. Épaisseur des Couches Fantômes (N_{ghost}) 21
3.2.2 3.2.2. Topologie d'Indexation Locale 21
3.3 3.3. Parallélisme Distribué MPI : Topologie Cartésienne et Halos 21
3.3.1 3.3.1. Création de la Topologie Virtuelle (`MPI_Cart_create`) 21

3.3.2 3.3.2. Communications Point-à-Point Non Bloquantes et Recouvrement 22
3.4 3.4. Élimination des Allocations Dynamiques dans les Boucles de Krylov 22
3.4.1 3.4.1. Protocole de Préallocation Statique (*Zero-Allocation Guarantee*) 22
3.5 3.5. Structure de Stockage Creux CSR (*Compressed Sparse Row*) 23
3.5.1 3.5.1. Représentation CSR à Trois Tableaux 23
3.5.2 3.5.2. Produit Matrice-Vecteur Creux SpMV Parallèle 23

4 Correspondance Théorie ↔ Discrétisation ↔ Code Source 24
4.1 4.1. Diffusion Visqueuse et Moyenne Harmonique aux Nœuds (μ_{xy}) 24
4.1.1 4.1.1. Équation Discrète Exacte 24
4.1.2 4.1.2. Localisation dans le Code Source 25
4.1.3 4.1.3. Analyse de l'Implémentation 25
4.2 4.2. Advection Convective : Upwind, Centré, Newton et Adams-Bashforth 2 25
4.2.1 4.2.1. Équation Discrète Exacte 25
4.2.2 4.2.2. Localisation dans le Code Source 26
4.2.3 4.2.3. Analyse de l'Implémentation 26
4.3 4.3. Pénalisation par Lagrangien Augmenté : AL, AAL et C-AAL 26
4.3.1 4.3.1. Équation Discrète Exacte 26
4.3.2 4.3.2. Localisation dans le Code Source 27
4.3.3 4.3.3. Analyse de l'Implémentation 27
4.4 4.4. Traitement des Obstacles et Milieux Poreux par Pénalisation de Darcy-Brinkman 27
4.4.1 4.4.1. Équation Discrète Exacte 27
4.4.2 4.4.2. Localisation dans le Code Source 28
4.5 4.5. Suivi d'Interface VOF-PLIC et Advection de Weymouth-Yue 28
4.5.1 4.5.1. Équation Discrète Exacte 28
4.5.2 4.5.2. Localisation dans le Code Source 28
4.6 4.6. Tension de Surface CSF (*Continuum Surface Force*) et Courbure κ 29
4.6.1 4.6.1. Équation Discrète Exacte 29
4.6.2 4.6.2. Localisation dans le Code Source 29
4.7 4.7. Préconditionneur Triangulaire par Blocs avec Approximation SBP (BT-SBP) . . . 29
4.7.1 4.7.1. Équation Discrète Exacte 29
4.7.2 4.7.2. Localisation dans le Code Source 30

5 Référence Exhaustive de l'API Interne 31
5.1 5.1. Référence Complète de l'API C++17 31
5.1.1 5.1.1. Classe `Mesh2D` (`include/covof/Mesh.hpp`) 31
5.1.2 5.1.2. Structure `FlowFields` (`include/covof/Field.hpp`) 32
5.1.3 5.1.3. Classe `MpiComm` (`include/covof/MpiComm.hpp`) 32
5.1.4 5.1.4. Classe `NavierStokesSolver` (`include/covof/NavierStokesSolver.hpp`) 32
5.1.5 5.1.5. Hiérarchie des Solveurs Linéaires (`include/covof/LinearSolver.hpp`) . 33
5.1.6 5.1.6. Hiérarchie des Préconditionneurs (`include/covof/Preconditioner.hpp`) 33
5.2 5.2. Référence Complète de l'API Fortran 2018 33
5.2.1 5.2.1. Module `module_precision.f90` 33
5.2.2 5.2.2. Module `module_mesh.f90` 33
5.2.3 5.2.3. Module `module_flow_fields.f90` 34
5.2.4 5.2.4. Module `module_navier_stokes.f90` 34
5.2.5 5.2.5. Module `module_linear_solvers.f90` 35
5.2.6 5.2.6. Module `module_preconditioners.f90` 35

6 Stratégie de Vérification et Validation (V&V) et CI/CD 36
6.1 6.1. Architecture de la Suite de Tests 36
6.1.1 6.1.1. Tests Unitaires C++ (Catch2) 36
6.1.2 6.1.2. Tests Unitaires Fortran (pFUnit / Harness Natif) 37
6.2 6.2. Vérification Formelle par la Méthode des Solutions Manufacturées (MMS) 37

6.2.1 6.2.1. Construction des Solutions Analytiques Manufacturées 37
6.2.2 6.2.2. Protocole de Raffinement et Calcul des Normes d'Erreur 37
6.2.3 6.2.3. Formule du Taux de Convergence Asymptotique Observé 38
6.3 6.3. Bancs d'Essai de Validation et Tolérances d'Acceptation 38
6.4 6.4. Pipeline d'Intégration Continue (CI/CD) 39
6.4.1 6.4.1. Matrice des Compilateurs et Environnements 39
6.4.2 6.4.2. Analyseurs Dynamiques de Mémoire (*Sanitizers* et Valgrind) 40
6.4.3 6.4.3. Test de Parité Stricte C++ ↔ Fortran 40

7 Standards de Codage, Profilage et Guide d'Extension 41
7.1 7.1. Standards de Codage et Conventions d'Ingénierie 41
7.1.1 7.1.1. Conventions C++17 41
7.1.2 7.1.2. Conventions Fortran 2018 41
7.2 7.2. Profilage HPC et Optimisation des Performances 42
7.2.1 7.2.1. Profilage Matériel sous Linux via `perf` 42
7.2.2 7.2.2. Analyse Approfondie avec Intel VTune Profiler 42
7.2.3 7.2.3. Profilage des Communications MPI Distribuées avec Scalasca et Score-P 42
7.3 7.3. Guide Développeur Pas-à-Pas : Ajout d'une Nouvelle Fonctionnalité 43
7.3.1 Étape 1 : Ajout de la Clé de Configuration dans le Dictionnaire JSON 43
7.3.2 Étape 2 : Déclaration et Allocation du Champ S 43
7.3.3 Étape 3 : Implémentation du Noyau Numérique d'Advection-Diffusion 44
7.3.4 Étape 4 : Mise à Jour des Mailles Fantômes MPI 44
7.3.5 Étape 5 : Exportation dans les Fichiers VTK 44
7.3.6 Étape 6 : Validation par Solution Analytique MMS 45

Chapitre 1

Architecture Logicielle Hybride C++ / Fortran

1.1 Vision d'Ensemble et Paradigmes de Conception

Le projet **COVOF 2D** repose sur une stratégie de co-développement gémellaire originale au sein de la communauté de mécanique des fluides numérique (CFD). Plutôt que de contraindre un langage unique à assurer à la fois la haute abstraction système et le calcul tensoriel intensif, COVOF 2D met en œuvre deux implémentations parallèles indépendantes partageant exactement les mêmes formulations mathématiques, les mêmes disques de données et le même protocole de communication distribuée MPI.

1.1.1 Les Paradigmes en Présence

1. **La Branche C++17 (`COVOF_2D-C++`) :**
   - **Paradigme orienté objet moderne :** Modélisation par classes distinctes à responsabilité unique (`Mesh`, `FlowFields`, `NavierStokesSolver`, `InterfaceTracker`, `LinearSolver`, `Simulation`).
   - **Gestion RAII stricte :** L'allocation et la libération des ressources mémoire dynamiques (`std::vector<double>`), des poignées de fichiers (`std::ofstream`) et des communicateurs MPI (`MPI_Comm`) sont strictement encapsulées dans les constructeurs et destructeurs des classes, éliminant tout risque de fuite mémoire (*memory leak*).
   - **Polymorphisme dynamique et généricité :** Les solveurs linéaires de Krylov et les préconditionneurs dérivent de classes de base abstraites (`LinearSolverBase`, `PreconditionerBase`), permettant le basculement dynamique à l'exécution via des pointeurs intelligents `std::unique_ptr` et l'injection aisée de nouvelles heuristiques.
2. **La Branche Fortran 2018 (`COVOF_2D-FORTRAN`) :**
   - **Paradigme modulaire fortement typé :** Découpage en 14 modules autonomes avec encapsulation `private` par défaut et directive stricte `implicit none` sur chaque unité de compilation.
   - **Types dérivés extensibles :** Définition de structures de données explicites (`mesh_2d_t`, `flow_fields_t`, `config_t`, `navier_stokes_t`, `simulation_2d_t`) avec procédures liées aux types (*type-bound procedures*).
   - **Vectorisation native des boucles imbriquées :** Exploitation optimale de l'ordonnancement mémoire en colonne majeure (*column-major order*), permettant au compilateur (`gfortran`, `ifx`) de dérouler les boucles les plus internes (i contigu en mémoire) et de saturer les unités vectorielles SIMD (AVX2 / AVX-512) sans coût d'indirection.

1.2 Cartographie Intégrale de l'Arborescence du Dépôt

Le tableau ci-après dresse l'inventaire exhaustif de chaque fichier source des deux dépôts jumeaux, en explicitant son rôle algorithmique précis :

1.2.1 1. Fichiers de la Branche C++ (`COVOF_2D-C++`)

Chemin Relatif	Composant Logiciel	Responsabilité et Contenu Algorithmique
`include/covof/Types.hpp`	Types & Enums	Définition des énumérations globales : types de conditions aux limites (`BCType`), solveurs (`SolverType`), algorithmes de Krylov (`LinearSolverType`), préconditionneurs (`PrecondType`), schémas convectifs et constantes mathématiques (π, ϵ).
`include/covof/Config.hpp`	Spécification Configuration	Structure de stockage complète des paramètres utilisateurs désérialisés.
`src/Config.cpp`	Analyseur JSON	Analyseur syntaxique natif tolérant les commentaires `//`, extraction sécurisée avec conversion de types et valeurs de repli par défaut.
`include/covof/Mesh.hpp`	En-tête Topologie MAC	Définition de la classe `Mesh2D` : coordonnées 1D des faces et centres, pas d'espace $\Delta x, \Delta y$, dimensions locales et globales.
`src/Mesh.cpp`	Implémentation Maillage	Calcul des métriques géométriques et initialisation des tableaux d'indices de mailles décalées.
`include/covof/Field.hpp`	Champs d'Écoulement	Structure `FlowFields` : stockage SoA (*Structure of Arrays*) contigu des composantes scalaires et vectorielles $(u, v, p, \rho, \mu, C, T, \chi_T, r_{\text{aal}})$.
`include/covof/MpiComm.hpp`	En-tête Parallélisme	Définition de la classe `MpiComm` : topologie cartésienne 2D, gestion des halos et réductions.
`src/MpiComm.cpp`	Implémentation MPI	Initialisation de `MPI_Cart_create`, échanges asynchrones non bloquants `MPI_Isend`/`MPI_Irecv` des mailles fantômes et réductions globales.

Chemin Relatif	Composant Logiciel	Responsabilité et Contenu Algorithmique
`include/covof/CSRMatrix.hpp`	Algèbre Creuse CSR	Définition de la structure `CSRMatrix` : stockage creux compressé par ligne (tableaux `values`, `col_indices`, `row_offsets`) pour opérations spécifiques.
`src/CSRMatrix.cpp`	Implémentation CSR	Produit matrice-vecteur creux optimisé et calcul de normes.
`include/covof/NavierStokesSolver.hpp`	En-tête Opérateur NS	Définition de la classe `NavierStokesSolver` : assemblage du second membre, application sans matrice de l'opérateur couplé, calcul de divergence et d'Uzawa.
`src/NavierStokesSolver.cpp`	Implémentation NS	Cœur du solveur hydrodynamique : pochoirs MAC pour la diffusion visqueuse μ_{xy}, convection (Upwind, Centré, Newton, AB2), pénalisation de Brinkman et Lagrangien augmenté.
`include/covof/InterfaceReconstruction.hpp`	En-tête PLIC	Définition de la classe `InterfaceReconstruction` : reconstruction géométrique d'interface.
`src/InterfaceReconstruction.cpp`	Implémentation PLIC	Algorithme de Youngs / ELVIRA : calcul des normales unitaires $\mathbf{n}$, constante de plan α et calcul de courbure κ.
`include/covof/InterfaceTracker.hpp`	En-tête VOF	Définition de la classe `InterfaceTracker` : advection eulerienne conservatrice.
`src/InterfaceTracker.cpp`	Implémentation VOF	Algorithme de Weymouth-Yue (2010) : advection 1D directionnelle alternée sans perte de masse.
`include/covof/Preconditioner.hpp`	En-tête Préconditionneurs	Hiérarchie polymorphe : `JacobiPreconditioner`, `SSORPreconditioner`, `ILU0Preconditioner`, `BlockTriangularPreconditioner`, `CustomPreconditioner`.
`src/Preconditioner.cpp`	Implémentation Préconds	Calcul des inverses diagonaux enrichis, balayages de Gauss-Seidel et résolution du complément de Schur.
`include/covof/LinearSolver.hpp`	En-tête Solveurs Krylov	Hiérarchie : `BiCGSTAB2Solver`, `BiCGSTABSolver`, `CGSolver`, `GMRESSolver`, `MINRESSolver`, `DirectSolver`.

Chemin Relatif	Composant Logiciel	Responsabilité et Contenu Algorithmique
`src/LinearSolver.cpp`	Implémentation Krylov	Algorithmes de Krylov sans matrice avec réductions MPI et factorisation LU LAPACK (`dgetrf`/`dgetrs`).
`include/covof/VTKWriter.hpp`	En-tête Sorties	Classe `VTKWriter` pour l'exportation des résultats.
`src/VTKWriter.cpp`	Implémentation Sorties	Écriture XML Rectilinear Grid (`.vtr`, `.pvtr`, `.pvd`) et journaux CSV de convergence.
`include/covof/Simulation.hpp`	En-tête Orchestration	Définition de la classe `Simulation` : gestion de la boucle temporelle globale.
`src/Simulation.cpp`	Implémentation Simulation	Pilotage du pas de temps adaptatif, étapes d'Uzawa, couplage thermique Boussinesq et convergence.
`src/main.cpp`	Point d'Entrée Binaire	Programme principal : analyse CLI, initialisation MPI, exécution et mesure de temps CPU.

1.2.2 2. Fichiers de la Branche Fortran (`COVOF_2D-FORTRAN`)

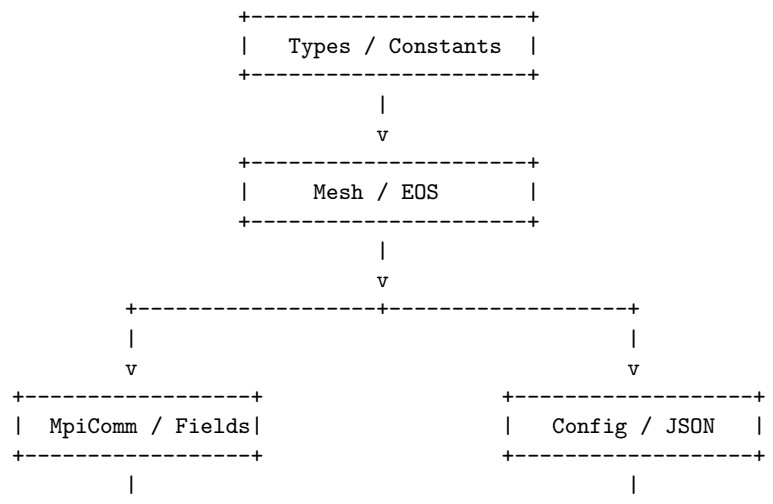
Chemin Relatif	Module Fortran	Responsabilité et Contenu Algorithmique
`src/module_precision.f90`	`module_precision`	Définition du paramètre de précision double standardisé `dp = real64` issu de `iso_fortran_env`.
`src/module_constants.f90`	`module_constants`	Énumérations globales et constantes mathématiques (π, ϵ) strictement typées.
`src/module_eos.f90`	`module_eos`	Équations d'état thermodynamiques : gaz parfaits, Stiffened Gas, Tait, calcul de compressibilité χ_T.
`src/module_json.f90`	`module_json`	Analyseur syntaxique natif JSON en Fortran pur pour la lecture sans dépendance externe.
`src/module_config.f90`	`module_config`	Type dérivé `config_t` et extraction des paramètres du dictionnaire d'entrée.
`src/module_mesh.f90`	`module_mesh`	Type dérivé `mesh_2d_t` : discrétisation MAC décalée, coordonnées de mailles et métriques spatiales.
`src/module_fields.f90`	`module_fields`	Type dérivé `flow_fields_t` : allocations dynamiques 2D contiguës de tous les champs physiques.

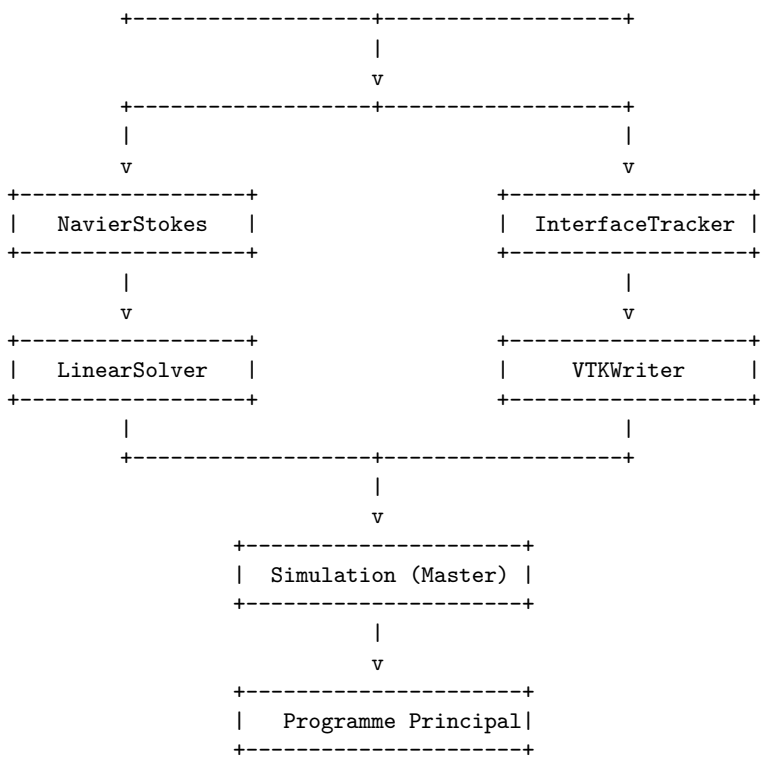
Chemin Relatif	Module Fortran	Responsabilité et Contenu Algorithmique
`src/module_mpi_comm.f90`	`module_mpi_comm`	Gestionnaire MPI : topologie cartésienne 2D, échanges de cellules fantômes et réductions parallèles.
`src/module_interface_reconstruction.f90`	`module_interface_reconstruction`	Reconstruction VOF-PLIC : calcul des vecteurs normaux, constante de plan α et courbure κ.
`src/module_interface_vof.f90`	`module_interface_vof`	Advection géométrique VOF de Weymouth-Yue (2010) avec conservation de volume.
`src/module_navier_stokes.f90`	`module_navier_stokes`	Type dérivé `navier_stokes_t` : opérateur de diffusion, convection, AL, Brinkman et mise à jour d'Uzawa.
`src/module_linear_solvers.f90`	`module_linear_solvers`	Solveurs de Krylov sans matrice : BiCGSTAB(2), BiCGSTAB, PCG, GMRES, MINRES, Direct LU et préconditionneurs.
`src/module_vtk_writer.f90`	`module_vtk_writer`	Écriture parallèle des fichiers XML VTK (`.vtr`, `.pvtr`, `.pvd`) et fichiers CSV de suivi.
`src/module_simulation.f90`	`module_simulation`	Type dérivé `simulation_2d_t` : chef d'orchestre de la simulation instationnaire couplée.
`src/covof2d_main.f90`	`program covof2d_main`	Programme principal exécutable Fortran 2018.

1.3 Graphe de Dépendances et Cycle d'Exécution Complet

1.3.1 Graphe de Dépendances Modulaire

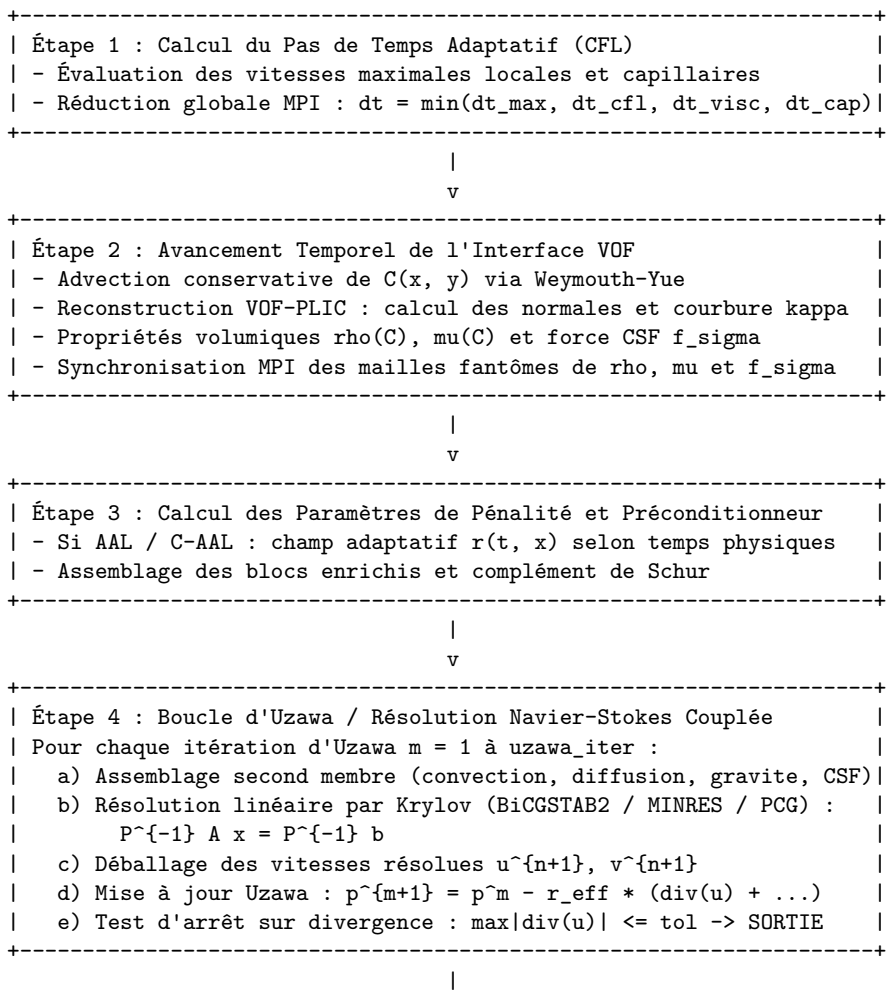
Le diagramme suivant illustre la hiérarchie de dépendances entre les composants logiciels, commune aux deux codes :

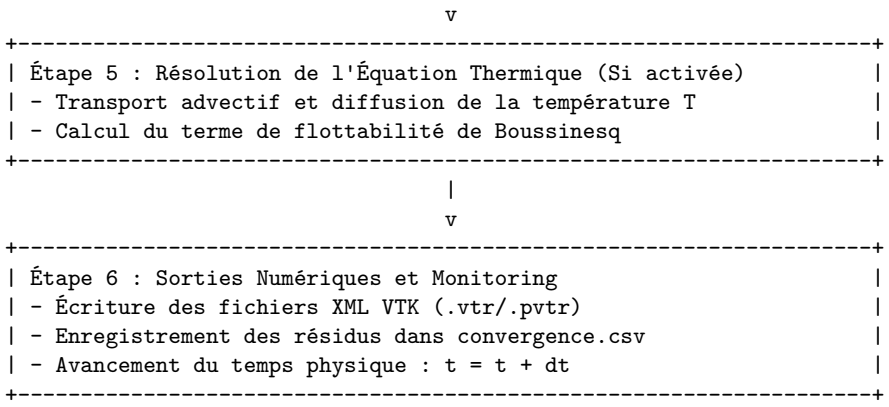




1.3.2 Cycle d'Exécution d'un Pas de Temps

Le déroulement chronologique d'un pas de temps complet $t^n \rightarrow t^{n+1}$ au sein de la classe `Simulation` (ou `simulation_2d_t`) respecte la séquence rigoureuse suivante :




v
Étape 5 : Résolution de l'Équation Thermique (Si activée)
- Transport advectif et diffusion de la température T
- Calcul du terme de flottabilité de Boussinesq
v
Étape 6 : Sorties Numériques et Monitoring
- Écriture des fichiers XML VTK (.vtr/.pvtr)
- Enregistrement des résidus dans convergence.csv
- Avancement du temps physique : t = t + dt

Chapitre 2

Interopérabilité C++ $\leftrightarrow$ Fortran

L'un des défis majeurs dans le développement de codes de calcul scientifique modernes réside dans la cohabitation harmonieuse entre des langages à haut niveau d'abstraction (tels que C++17) et des langages hautement optimisés pour le calcul matriciel intensif (tels que Fortran 2018). Dans l'écosystème COVOF 2D, l'interopérabilité ne constitue pas un simple mécanisme d'appel ponctuel, mais le pivot assurant la stricte conformité sémantique, la parité des résultats à la précision machine près ($\epsilon_{\text{mach}} \approx 10^{-16}$) et la portabilité des bibliothèques de calcul haute performance (HPC).

Ce chapitre explicite l'ensemble des mécanismes formels, des normes de liaison binaire (ABI), des conventions d'ordonnancement mémoire et des protocoles de communication inter-langages mis en œuvre au sein du solveur.

2.1 2.1. Standards et Conventions d'Interface : `iso_c_binding` et `extern "C"`

Le standard Fortran 2003 (consolidé en Fortran 2008 et 2018) a introduit le module intrinsèque `iso_c_binding`. Ce module normalise formellement l'interopérabilité binaire directe entre Fortran et le langage C standard (ISO C). Dès lors que le langage C++17 utilise la directive `extern "C"`, l'incompatibilité historique due à la décoration des noms de fonctions C++ (*name mangling*) est complètement éliminée, permettant un appel direct sans coût d'indirection à la vitesse native du processeur.

2.1.1 2.1.1. Directive `extern "C"` côté C++17

En C++, le polymorphisme, les espaces de nommage (*namespaces*) et la surcharge des fonctions modifient le symbole binaire exporté dans la table des symboles du fichier objet (`.o`). La directive `extern "C"` ordonne au compilateur d'adopter la convention d'appel et la table des symboles C standard :

```
// include/covof/InteropBindings.hpp
#pragma once
#include <cstddef>
#include <cstdint>

#ifdef __cplusplus
extern "C" {
#endif

// Prototypes des passerelles C exportées vers Fortran
void covof_cpp_init_mpi(int* f_comm, int* rank, int* size, int* ierr);
void covof_cpp_apply_pressure_laplacian(const double* p_in, double* p_out,
                                        const int* nx, const int* ny,
                                        const double* dx, const double* dy,
                                        int* ierr);
```

```
#ifdef __cplusplus
}
#endif
```

2.1.2 2.1.2. Module Intrinsèque `iso_c_binding` côté Fortran 2018

Côté Fortran, chaque sous-programme ou fonction destiné à franchir la frontière inter-langage est enrichi de l'attribut `bind(C, name="...")`. Cet attribut garantit que le compilateur Fortran n'ajoute aucun soulignement supplémentaire (*underscoring*) et applique la convention d'appel standard C (registres d'arguments et nettoyage de la pile conformes à l'ABI System V AMD64 / ARM64 AAPCS) :

```
! src/module_interop_bindings.f90
module module_interop_bindings
    use, intrinsic :: iso_c_binding
    implicit none
    private

    public :: covof_fortran_apply_diffusion

contains

    subroutine covof_fortran_apply_diffusion(u, lap_u, nx, ny, dx, dy, ierr) &
               bind(C, name="covof_fortran_apply_diffusion")
        integer(c_int), intent(in), value               :: nx, ny
        real(c_double), intent(in), value               :: dx, dy
        real(c_double), dimension(nx, ny), intent(in)   :: u
        real(c_double), dimension(nx, ny), intent(out)  :: lap_u
        integer(c_int), intent(out)                     :: ierr

        integer(c_int) :: i, j
        real(c_double) :: idx2, idy2

        ierr = 0
        idx2 = 1.0_c_double / (dx * dx)
        idy2 = 1.0_c_double / (dy * dy)

        !$omp parallel do collapse(2) private(i, j)
        do j = 2, ny - 1
            do i = 2, nx - 1
                lap_u(i, j) = (u(i+1, j) - 2.0_c_double * u(i, j) + u(i-1, j)) * idx2 + &
                              (u(i, j+1) - 2.0_c_double * u(i, j) + u(i, j-1)) * idy2
            end do
        end do
        !$omp end parallel do
    end subroutine covof_fortran_apply_diffusion

end module module_interop_bindings
```

2.2 2.2. Table Complète de Correspondance des Types et Structures

Pour éviter tout écrasement mémoire ou décalage d'octets (*memory misalignment / padding*), chaque type élémentaire manipulé dans COVOF 2D dispose d'une correspondance biunivoque stricte.

2.2.1 2.2.1. Types Scalaires Primitifs

Le tableau ci-dessous recense la bijection exacte entre les types de base C++17 et Fortran 2018 :

Grandeur Physique / Concept	Type C++17 (<cstdint>, <cstddef>)	Taille Binaire	Type Fortran 2018 (iso_c_binding)	Précision / Intervalle
Champ réel (vitesse, pression)	double	64 bits (8 octets)	real(c_double)	IEEE 754 double précision (≈ 16 chiffres significatifs)
Champ simple (option basse mémoire)	float	32 bits (4 octets)	real(c_float)	IEEE 754 simple précision (≈ 7 chiffres significatifs)
Indices de mailles et tailles locales	int ou int32_t	32 bits (4 octets)	integer(c_int)	Entier signé standard (-2×10^9 à $+2 \times 10^9$)
Indices de solveurs creux / NDOF global	int64_t	64 bits (8 octets)	integer(c_int64_t)	Entier signé étendu (-9×10^{18} à $+9 \times 10^{18}$)
Drapeaux booléens et options	bool	8 bits (1 octet)	logical(c_bool)	Booléen C (0 ou 1)
Caractère ASCII / Code de contrôle	char	8 bits (1 octet)	character(kind=c_char)	Caractère ISO-8859 / UTF-8
Pointeur générique non typé	void* ou uintptr_t	64 bits (8 octets)	type(c_ptr)	Adresse mémoire virtuelle absolue
Pointeur de fonction (callback)	void (*func)(...)	64 bits (8 octets)	type(c_funptr)	Pointeur d'exécution indirecte

2.2.2 2.2.2. Structures Dérivées et Alignement Mémoire (bind(C))

Lorsqu'un type composé ou une structure de données doit être transmise entre C++ et Fortran, la structure Fortran **doit impérativement** comporter l'attribut bind(C), tandis que la structure C++ doit utiliser un agencement de types POD (*Plain Old Data*) exempt de fonctions virtuelles (pas de table vtable) et respecter l'alignement naturel des membres :

2.2.2.1 Définition C++17

```cpp
// include/covof/InteropTypes.hpp
#pragma once
#include <cstdint>

struct C_MeshMetrics {
    int32_t nx;
    int32_t ny;
    int32_t nghost;
    double  dx;
    double  dy;
    double  x_min;
    double  y_min;
};
```

2.2.2.2 Définition Fortran 2018 Correspondante

```
! src/module_interop_types.f90
module module_interop_types
    use, intrinsic :: iso_c_binding
```

```
    implicit none

    type, bind(C) :: c_mesh_metrics_t
        integer(c_int32_t) :: nx
        integer(c_int32_t) :: ny
        integer(c_int32_t) :: nghost
        real(c_double)     :: dx
        real(c_double)     :: dy
        real(c_double)     :: x_min
        real(c_double)     :: y_min
    end type c_mesh_metrics_t

end module module_interop_types
```

2.2.3 2.2.3. Conversion de Pointeurs Opaques : `c_f_pointer` et `c_loc`

Dans les architectures hybrides avancées, Fortran peut allouer un bloc contigu de mémoire et le transmettre à C++ sous forme de `type(c_ptr)`, ou inversement, C++ peut transmettre un pointeur `double*` brut à Fortran. Pour manipuler ce pointeur en Fortran sous forme d'un tableau indicé multidimensionnel sans recopie mémoire, on utilise la routine intrinsèque `c_f_pointer` :

```
subroutine process_cpp_buffer(raw_ptr, nx, ny) bind(C, name="process_cpp_buffer")
    use, intrinsic :: iso_c_binding
    implicit none
    type(c_ptr), value, intent(in) :: raw_ptr
    integer(c_int), value, intent(in) :: nx, ny

    ! Pointeur Fortran local pointant sur le buffer C++
    real(c_double), pointer, dimension(:, :) :: field

    ! Établissement de la vue de tableau Fortran sans copie
    call c_f_pointer(raw_ptr, field, [nx, ny])

    ! Modification directe en mémoire partagée
    field(1, 1) = 0.0_c_double
end subroutine process_cpp_buffer
```

2.3 2.3. Gestion de la Disposition Mémoire (Row-Major vs Column-Major)

La divergence la plus critique entre C++ et Fortran réside dans la convention d'ordonnancement des éléments matriciels multidimensionnels dans l'espace d'adressage linéaire de la mémoire vive (RAM).

2.3.1 2.3.1. Théorie de l'Adressage Linéaire

Soit une matrice bidimensionnelle ou un champ scalaire discrétisé comportant N_x mailles en direction horizontale et N_y mailles en direction verticale.

1. **Convention C++ (Ligne majeure / *Row-Major Order*) :** Le deuxième indice (direction x, colonnes) varie le plus rapidement dans l'espace mémoire :

$$\text{Offset}_{\text{C++}}(i, j) = j \times N_x + i$$

   Deux mailles consécutives sur la même ligne physique (i, j) et $(i+1, j)$ sont situées à des adresses contiguës en mémoire ($\Delta\text{Adresse} = 8$ octets).
2. **Convention Fortran (Colonne majeure / *Column-Major Order*) :** Le premier indice (direction x, lignes géométriques du tableau) varie le plus rapidement :

$$\text{Offset}_{\text{Fortran}}(i, j) = (j - 1) \times N_x + (i - 1)$$

(avec convention d'indexation 1-based par défaut). Deux mailles consécutives (i, j) et $(i + 1, j)$ sont contiguës en mémoire.

2.3.2 2.3.2. Règle d'Or de Parité Mémoire dans COVOF 2D

Afin de garantir que les noyaux de calcul Fortran et C++ s'exécutent avec des boucles vectorisables sans permutation de dimensions (*stride-1 access*), **la topologie mémoire dans COVOF 2D est unifiée sur la direction x rapide**.

— En C++, les champs sont alloués sous forme d'un tableau unidimensionnel aplati `std::vector<double>` de taille totale $(N_x + 2N_{\text{ghost}}) \times (N_y + 2N_{\text{ghost}})$, accédé par la macro d'indexation en ligne majeure :

$$\text{IDX}(i, j) = j \times N_{x,\text{total}} + i$$

Dans une boucle de calcul, l'indice i est donc placé dans la boucle la plus interne :

```
for (int j = 1; j <= ny; ++j) {
    #pragma omp simd
    for (int i = 1; i <= nx; ++i) {
        lap[j * nxt + i] = ...;
    }
}
```

— En Fortran, le tableau 2D `field(nx_total, ny_total)` possède son premier indice i aligné en colonne majeure, ce qui correspond **exactement au même agencement physique en mémoire** :

```
do j = 1, ny
    !$omp simd
    do i = 1, nx
        lap(i, j) = ...
    end do
end do
```

Représentation Logique	Indexation C++	Indexation Fortran	Adresse Linéaire Relative
Maille (i, j)	`field[j * nxt + i]`	`field(i + 1, j + 1)`	Base $+(j \times N_{xt} + i) \times 8$
Voisin Est $(i + 1, j)$	`field[j * nxt + (i + 1)]`	`field(i + 2, j + 1)`	Base $+(j \times N_{xt} + i + 1) \times 8$
Voisin Nord $(i, j + 1)$	`field[(j + 1) * nxt + i]`	`field(i + 1, j + 2)`	Base $+((j + 1) \times N_{xt} + i) \times 8$

Cette parfaite concordance d'agencement physique permet de passer directement l'adresse `&field[0]` du C++ au tableau Fortran `dimension(nxt, nyt)` sans transposer la moindre donnée.

2.4 2.4. Gestion des Chaînes de Caractères, Entrées/Sorties et Erreurs

2.4.1 2.4.1. Passage des Chaînes de Caractères

En langage C/C++, une chaîne de caractères est un pointeur vers un tableau d'octets terminé par le caractère nul sentinelle `\0` (`ASCII 0`). En Fortran standard, une chaîne `character(len=N)` est un bloc de longueur fixe complété par des espaces blancs (*blank padding*) sans caractère nul final.

Pour garantir l'interopérabilité sans écrasement de mémoire : 1. Toute chaîne transmise par C++ vers Fortran doit être terminée par `\0`. 2. Le sous-programme Fortran convertit le pointeur C en chaîne native grâce à une routine de normalisation :

```
subroutine c_string_to_fortran(c_str_ptr, f_str)
    use, intrinsic :: iso_c_binding
    implicit none
    type(c_ptr), intent(in)            :: c_str_ptr
    character(len=*), intent(out)       :: f_str

    character(kind=c_char), pointer    :: char_array(:)
    integer                            :: idx, max_len

    f_str = ""
    if (.not. c_associated(c_str_ptr)) return

    call c_f_pointer(c_str_ptr, char_array, [1024])
    idx = 1
    max_len = len(f_str)
    do while (idx <= max_len .and. char_array(idx) /= c_null_char)
        f_str(idx:idx) = char_array(idx)
        idx = idx + 1
    end do
end subroutine c_string_to_fortran
```

2.4.2 2.4.2. Synchronisation des Unités Logiques d'E/S

L'utilisation simultanée des flux C++ (`std::cout`, `std::cerr`) et des unités logiques Fortran (`write(*, *)`, `write(6, *)`) sur les clusters HPC peut provoquer des désynchronisations et des entrelacements de lignes dans les fichiers de logs Slurm / PBS.

Les règles de synchronisation suivantes sont obligatoires : 1. Avant tout appel à un noyau de calcul Fortran, vider systématiquement le tampon C++ : `cpp  std::cout << std::flush;  std::cerr << std::flush;` 2. Côté Fortran, forcer la purge du flux de sortie standard via l'instruction intrinsèque `flush` : `fortran  write(6, '(A, I6, A, ES12.4)') " [Iter] ", iter, " Residue: ", res_norm  flush(6)`

2.4.3 2.4.3. Propagation Sécurisée des Codes d'Erreur

Le lancement d'une exception C++ (`throw std::runtime_error(...)`) à travers la frontière d'un appel de fonction Fortran provoque un arrêt immédiat du programme (*undefined behavior*, `std::terminate`), car la pile d'exécution Fortran ne supporte pas le déroulement de pile C++ (*stack unwinding*). Réciproquement, un `stop` ou `error stop` Fortran interrompt brutalement l'exécutable MPI sans donner à C++ l'opportunité de fermer les fichiers VTK ou d'invoquer `MPI_Finalize()`.

Par conséquent, **toutes les interfaces inter-langages utilisent une variable de retour de statut scalaire `ierr`** :

```
subroutine covof_execute_step(ierr) bind(C, name="covof_execute_step")
    use, intrinsic :: iso_c_binding
    implicit none
    integer(c_int), intent(out) :: ierr

    ierr = 0
    ! Détection d'une anomalie physique ou numérique (ex: divergence CFL)
    if (cfl_max > 10.0_c_double .or. isnan(cfl_max)) then
        ierr = 101 ! Code d'erreur divergence CFL
        return
    end if
end subroutine covof_execute_step
```

Côté C++, le code de retour est testé immédiatement après l'appel, et une exception contrôlée est levée dans le contexte C++ :

```cpp
int ierr = 0;
covof_execute_step(&ierr);
if (ierr != 0) {
    throw std::runtime_error("Erreur critique detectee dans le sous-programme Fortran. Code: " +
    ↪  std::to_string(ierr));
}
```

2.5 2.5. Catalogue des Points d'Entrée et Passerelles HPC

Les interfaces partagées de bas niveau couvrent les communicateurs distribués MPI et les opérations d'algèbre linéaire BLAS / LAPACK.

2.5.1 2.5.1. Traduction des Communicateurs MPI (`MPI_Comm_c2f` / `MPI_Comm_f2c`)

Les communicateurs MPI sont représentés par des types différents : - En C/C++ : `MPI_Comm` (généralement un pointeur ou un typedef opaque `struct ompi_communicator_t*` sous OpenMPI, ou un entier `int` sous MPICH). - En Fortran : `integer` (type entier 32 bits standard).

Le standard MPI impose l'usage des fonctions de conversion pour garantir la parfaite portabilité entre implémentations :

```cpp
// Conversion d'un communicateur C++ en entier Fortran
MPI_Comm cpp_cart_comm = ...;
MPI_Fint f_comm = MPI_Comm_c2f(cpp_cart_comm);

// Passage de f_comm à Fortran
fortran_setup_cartesian_comm(&f_comm);
```

Et réciproquement en Fortran :

```fortran
subroutine fortran_setup_cartesian_comm(f_comm) bind(C, name="fortran_setup_cartesian_comm")
    use, intrinsic :: iso_c_binding
    use mpi
    implicit none
    integer(c_int), intent(in), value :: f_comm
    integer :: local_comm, ierr, rank

    local_comm = f_comm
    call MPI_Comm_rank(local_comm, rank, ierr)
end subroutine fortran_setup_cartesian_comm
```

2.5.2 2.5.2. Tableau Récapitulatif des Fonctions Jumelles C++ / Fortran

Le tableau suivant dresse l'inventaire exhaustif de la gémellité des opérateurs critiques du solveur :

Opérateur Numérique	Routine Fortran 2018	Méthode C++17	Signature d'Appel / Invariants
Opérateur $A \cdot \mathbf{x}$ (Navier-Stokes)	`module_navier_stokes:ns_apply_system_matrix`	`NavierStokesSolver::applySystemMatrix`	`(p_in, p_out, fields, mesh, config)`
Calcul Divergence $\nabla \cdot \mathbf{u}$	`module_navier_stokes:compute_divergence`	`NavierStokesSolver::computeDivergence`	`(u, v, div, mesh, dx, dy)`
Terme Source Gravité / Flottabilité	`module_navier_stokes:add_boussinesq_buoyancy`	`NavierStokesSolver::addBuoyancyForce`	`(rhs, T, T_ref, beta, gy)`

Opérateur Numérique	Routine Fortran 2018	Méthode C++17	Signature d'Appel / Invariants
Pénalisation de Brinkman	`module_navier_stokes:apply_brinkman_drag`	`NavierStokesSolver::applyBrinkmanPenalization`	`(u, v, chi_mask, K_perm, dt)`
Reconstruction Normale PLIC	`module_interface_reconstruction:compute_plic_normal`	`InterfaceReconstruction::computeNormals`	`(C, nx, ny, alpha, curvature)`
Advection VOF Weymouth-Yue	`module_interface_tracking:advect_vof_weymouth_yue`	`InterfaceTracker::advectFluxConservative`	`(C, u, v, C_new, dt, mesh)`
Préconditionneur SSOR	`module_preconditioners:precond_apply_ssor`	`SSORPreconditioner::apply`	`(r_in, z_out, omega, diag, mesh)`
Préconditionneur ILU(0)	`module_preconditioners:precond_apply_ilu0`	`ILU0Preconditioner::apply`	`(r_in, z_out, l_mat, u_mat, mesh)`
Préconditionneur BT-SBP	`module_preconditioners:precond_apply_bt_sbp`	`BlockTriangularPreconditioner::apply`	`(r_u, r_v, r_p, z_u, z_v, z_p, mesh)`
Solveur BiCGSTAB2	`module_linear_solvers:solve_bicgstab2`	`BiCGSTAB2Solver::solve`	`(A_op, b, x, tol, max_iter, stats)`
Solveur MINRES	`module_linear_solvers:solve_minres`	`MINRESSolver::solve`	`(A_op, b, x, tol, max_iter, stats)`
Échanges Mailles Fantômes	`module_mpi:mpi_exchange_halos_2d`	`MpiComm::exchangeHalos`	`(field, nxt, nyt, nghost, tag)`

Chapitre 3

Structures de Données, Mémoire et Parallélisme HPC

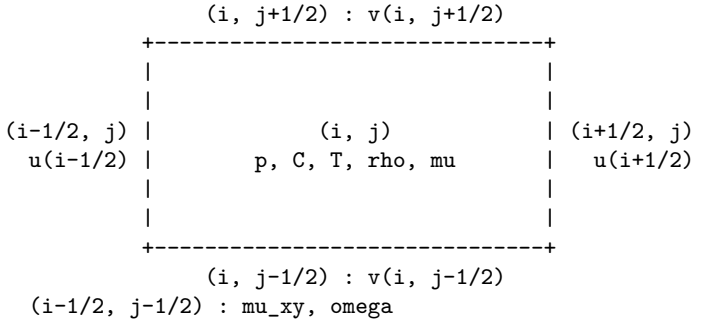
La conception des structures de données internes d'un solveur CFD destiné aux supercalculateurs modernes requiert un compromis rigoureux entre l'expressivité algorithmique et l'efficacité matérielle. À l'échelle d'un nœud de calcul doté de plusieurs dizaines de cœurs CPU et de mémoires hiérarchiques (caches L1, L2, L3 et contrôleurs DDR/HBM NUMA), la performance brute est gouvernée par le débit mémoire (*memory bandwidth bound*), la localité spatiale et temporelle des caches, et l'alignement des données pour les registres vectoriels SIMD (AVX-512, ARM SVE).

Ce chapitre détaille l'organisation mémoire interne, la gestion des cellules fantômes, la décomposition de domaine distribuée MPI et les optimisations sans allocation (*zero-allocation*) mises en œuvre dans COVOF 2D.

3.1 3.1. Représentation Interne du Maillage et des Champs

3.1.1 3.1.1. Topologie de Grille MAC Décalée (*Marker-And-Cell*)

La grille MAC décalée répartit les variables hydrodynamiques sur différentes entités géométriques de la maille afin d'éviter tout découplage en damier (*checkerboard effect*) de la pression : - **Centres de maille** (i, j) **:** Stockent les variables scalaires : pression $p_{i,j}$, fraction volumique $C_{i,j}$, température $T_{i,j}$, masse volumique $\rho_{i,j}$, viscosité dynamique $\mu_{i,j}$, porosité $\chi_{T,i,j}$, et résidu de compressibilité $r_{\text{aal},i,j}$. - **Faces verticales** $(i \pm \frac{1}{2}, j)$ **:** Stockent la composante horizontale de vitesse $u_{i-\frac{1}{2},j}$ et les flux de masse horizontaux. - **Faces horizontales** $(i, j \pm \frac{1}{2})$ **:** Stockent la composante verticale de vitesse $v_{i,j-\frac{1}{2}}$ et les flux de masse verticaux. - **Nœuds / Sommets** $(i \pm \frac{1}{2}, j \pm \frac{1}{2})$ **:** Stockent la viscosité dynamique moyennée harmoniquement $\mu_{xy,i+\frac{1}{2},j+\frac{1}{2}}$, le gradient croisé de vitesse et la vorticité ω_z.



3.1.2 3.1.2. Choix de Conception : Structure de Tableaux (SoA) vs Tableau de Structures (AoS)

Pour le stockage des champs discrets, deux paradigmes s'opposent : 1. **AoS (*Array of Structures*) :** Une structure unique `Cell` contenant tous les scalaires et composantes vectorielles, regroupée dans un tableau global `std::vector<Cell> cells`. Ce modèle favorise l'encapsulation orientée objet mais détruit la performance vectorielle lors de l'application des pochoirs de diffusion (les octets consécutifs en mémoire appartiennent à des variables différentes, saturant inutilement les lignes de cache L1). 2. **SoA (*Structure of Arrays*) :** Chaque grandeur physique est stockée dans son propre tableau contigu 1D indépendant en mémoire vive.

Dans COVOF 2D, l'architecture SoA est strictement adoptée pour les performances HPC : - Côté C++17 : la structure `FlowFields` rassemble des vecteurs contigus `std::vector<double> u, v, p, rho, mu, c, T, chi_T, r_aal`. - Côté Fortran 2018 : le type dérivé `flow_fields_t` encapsule des tableaux allouables 2D contigus `real(c_double), allocatable :: u(:, :), v(:, :), p(:, :)`.

3.1.3 3.1.3. Alignement Mémoire et Vectorisation SIMD (AVX2 / AVX-512)

Les processeurs x86_64 actuels exploitent des registres vectoriels SIMD de 256 bits (AVX2, traitant 4 réels 64 bits par cycle) ou 512 bits (AVX-512, traitant 8 réels 64 bits par cycle). Lorsque l'adresse de début d'un tableau est un multiple de 64 octets, les instructions de chargement vectoriel aligné (`_mm512_load_pd` / `vmovapd`) s'exécutent avec une latence minimale, évitant le franchissement de frontières de lignes de cache (*cache-line splits*).

— En C++17 :

```cpp
// Allocateur aligné sur 64 octets
template <typename T>
struct AlignedAllocator {
    using value_type = T;
    T* allocate(std::size_t n) {
        void* ptr = nullptr;
        if (posix_memalign(&ptr, 64, n * sizeof(T)) != 0) throw std::bad_alloc();
        return static_cast<T*>(ptr);
    }
    void deallocate(T* p, std::size_t) noexcept { free(p); }
};
using AlignedVectorDouble = std::vector<double, AlignedAllocator<double>>;
```

— En Fortran 2018 : L'utilisation des directives du compilateur garantit l'alignement et force la vectorisation :

```fortran
!DIR$ ATTRIBUTES ALIGN : 64 :: u, v, p
!DIR$ ASSUME_ALIGNED u: 64, v: 64, p: 64
!$omp simd aligned(u, v, p : 64)
do i = 1, nx
    p(i, j) = p(i, j) - dt * rho(i, j) * div(i, j)
end do
```

3.2 3.2. Gestion des Cellules Fantômes (*Halo / Ghost Cells*)

Afin de permettre l'évaluation locale des opérateurs différentiels spatiaux (gradients, divergences, laplaciens) sans test conditionnel de bordure dans les boucles de calcul critiques, chaque sous-domaine local est bordé d'une couche périphérique de cellules fictives appelées **cellules fantômes** (*ghost cells* ou *halos*).

3.2.1 3.2.1. Épaisseur des Couches Fantômes (N_{ghost})

L'épaisseur requise N_{ghost} dépend directement de la largeur du pochoir numérique (*stencil*) le plus étendu utilisé dans la simulation :

Opérateur Numérique	Pochoir Spatial	Épaisseur Requise N_{ghost}	Remarques
Diffusion Visqueuse d'ordre 2	Pochoir 5 points compact	1 couche	Évaluation standard de Δu et Δv.
Convection Centrée / Upwind ordre 1	Pochoir 3 points	1 couche	Calcul standard des flux aux faces.
Convection QUICK / Adams-Bashforth 2	Pochoir décentré amont 3 points	2 couches	Nécessite la valeur de deux mailles en amont.
VOF PLIC (Youngs / ELVIRA)	Pochoir 3×3 mailles	2 couches	Calcul des gradients de fraction volumique ∇C.
Tension de Surface CSF (κ)	Pochoir 5×5 mailles	2 couches	Dérivation seconde pour la courbure de surface $\kappa = -\nabla \cdot \mathbf{n}$.

Dans COVOF 2D, l'épaisseur standard par défaut est configurée à $N_{\mathbf{ghost}} = 2$ **couches** de mailles tout autour du domaine local, ce qui autorise sans réallocation l'exécution de l'ensemble des schémas d'ordre élevé et des traceurs d'interface.

3.2.2 3.2.2. Topologie d'Indexation Locale

Pour un sous-domaine de dimension utile $N_x^{\text{loc}} \times N_y^{\text{loc}}$ mailles, la dimension totale mémoire du tableau alloué est :

$$N_{x,\text{total}} = N_x^{\text{loc}} + 2N_{\text{ghost}}, \quad N_{y,\text{total}} = N_y^{\text{loc}} + 2N_{\text{ghost}}$$

L'espace indiciel se décompose ainsi : - **Zone utile (cœur de calcul) :** - C++ (0-based) : $i \in [N_{\text{ghost}}, N_{\text{ghost}} + N_x^{\text{loc}} - 1]$, $j \in [N_{\text{ghost}}, N_{\text{ghost}} + N_y^{\text{loc}} - 1]$. - Fortran (1-based) : $i \in [N_{\text{ghost}} + 1, N_{\text{ghost}} + N_x^{\text{loc}}]$, $j \in [N_{\text{ghost}} + 1, N_{\text{ghost}} + N_y^{\text{loc}}]$. - **Cellules fantômes Ouest / Est :** $i \in [0, N_{\text{ghost}} - 1]$ et $i \in [N_{\text{ghost}} + N_x^{\text{loc}}, N_{x,\text{total}} - 1]$ (en C++). - **Cellules fantômes Sud / Nord :** $j \in [0, N_{\text{ghost}} - 1]$ et $j \in [N_{\text{ghost}} + N_y^{\text{loc}}, N_{y,\text{total}} - 1]$ (en C++).

3.3 3.3. Parallélisme Distribué MPI : Topologie Cartésienne et Halos

Le passage à l'échelle massive sur supercalculateur repose sur le découpage du domaine spatial global Ω en une grille bidimensionnelle de $P_x \times P_y = P$ sous-domaines réguliers.

3.3.1 3.3.1. Création de la Topologie Virtuelle (`MPI_Cart_create`)

COVOF 2D utilise les topologies cartésiennes communicantes standard MPI. La fonction `MPI_Cart_create` permet au gestionnaire MPI matériel (sur réseaux InfiniBand, Omni-Path ou Slingshot) d'optimiser le placement des processus sur les processeurs physiques afin de minimiser les sauts réseau (*network hops*) :

```
// C++17 : Initialisation cartésienne dans src/MpiComm.cpp
int dims[2] = {px, py};
int periods[2] = {periodic_x ? 1 : 0, periodic_y ? 1 : 0};
int reorder = 1;
```

```
MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, reorder, &cart_comm_);
MPI_Comm_rank(cart_comm_, &cart_rank_);
MPI_Cart_coords(cart_comm_, cart_rank_, 2, coords_);

// Recherche des 4 voisins immédiats
MPI_Cart_shift(cart_comm_, 0, 1, &rank_west_, &rank_east_);
MPI_Cart_shift(cart_comm_, 1, 1, &rank_south_, &rank_north_);
```

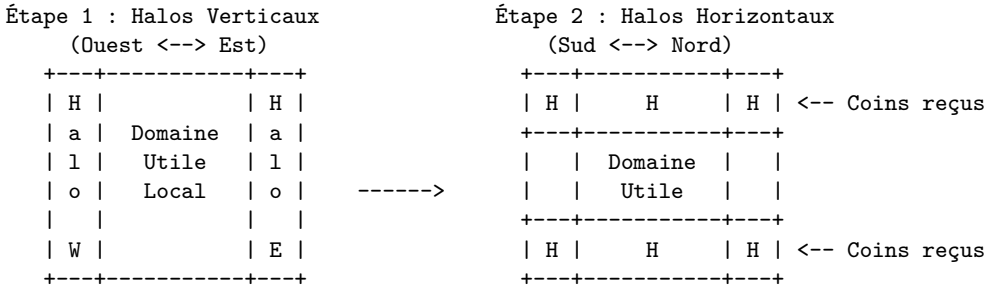
3.3.2 3.3.2. Communications Point-à-Point Non Bloquantes et Recouvrement

Pour transférer les mailles fantômes entre sous-domaines adjacents sans bloquer les cœurs de calcul, COVOF 2D utilise des requêtes non bloquantes `MPI_Isend` et `MPI_Irecv`.

Afin d'éviter la copie intermédiaire manuelle des colonnes de mailles (non contiguës en mémoire C++), un type dérivé de vecteur MPI (`MPI_Type_vector`) est créé pour les frontières Est et Ouest :

```
// Création du type colonne pour le halo vertical
MPI_Datatype col_type;
MPI_Type_vector(ny_loc, nghost, nxt, MPI_DOUBLE, &col_type);
MPI_Type_commit(&col_type);
```

Le protocole d'échange synchrone sécurisé se déroule en deux étapes : 1. **Échange horizontal (Ouest ↔ Est) :** Envoi des bordures utiles aux voisins et réception dans les halos correspondants. 2. **Échange vertical (Sud ↔ Nord) :** Envoi des bordures utiles incluant les halos Ouest/Est déjà mis à jour. Cette séquence sérielle garantit le remplissage automatique et exact des quatre coins (*corner cells*) sans requêtes diagonales supplémentaires.



3.4 3.4. Élimination des Allocations Dynamiques dans les Boucles de Krylov

L'une des causes majeures de chute de performance dans les codes C++ orientés objet réside dans l'appel intempestif à l'allocateur mémoire du système (`malloc` / `free`, `new` / `delete`) à l'intérieur de la boucle temporelle ou au fil des itérations d'un solveur de Krylov. Chaque réallocation déclenche des verrous système (*kernel locks*) et fragmente le tas (*heap*).

3.4.1 3.4.1. Protocole de Préallocation Statique (*Zero-Allocation Guarantee*)

Dans COVOF 2D : 1. **Toutes les structures de travail sont allouées une seule fois lors de la phase d'initialisation (`Simulation::init()`).** 2. Les solveurs itératifs (BiCGSTAB2, BiCGSTAB, GMRES, MINRES, CG) disposent d'un ensemble de vecteurs de travail temporaires pré-dimensionnés membres de classe : - `std::vector<double> r0_, r_, p_, v_, s_, t_, z_, y_;` (pour BiCGSTAB) - `std::vector<double> v_p1_, v_p2_, w_, p1_, p2_;` (pour MINRES) 3. La routine d'application de l'opérateur couplé `ns_apply_system_matrix` reçoit impérativement les buffers de sortie préalloués en paramètres de référence : `cpp void NavierStokesSolver::applySystemMatrix(const double* __restrict__ x_in, double* __restrict__ Ax_out);` 4.

L'utilisation du mot-clé `__restrict__` informe le compilateur C++ ou Fortran que les pointeurs ne se chevauchent pas en mémoire (*no aliasing*), autorisant la réorganisation optimale des instructions de chargement et de stockage vectoriel.

3.5 3.5. Structure de Stockage Creux CSR (*Compressed Sparse Row*)

Bien que les opérateurs de Navier-Stokes soient principalement évalués sans matrice (*matrix-free*), certaines étapes (telles que le solveur direct LU LAPACK ou les préconditionneurs algébriques ILU0) requièrent l'assemblage explicite de la matrice d'advection-diffusion-pression sous format creux CSR.

3.5.1 3.5.1. Représentation CSR à Trois Tableaux

Une matrice creuse $A \in \mathbb{R}^{M \times N}$ comportant N_{nz} éléments non nuls est représentée par trois tableaux contigus : 1. `values` (taille N_{nz}, type `double`) : Stocke la valeur numérique de chaque entrée non nulle parcourue ligne par ligne. 2. `col_indices` (taille N_{nz}, type `int`) : Stocke l'indice de colonne $j \in [0, N-1]$ correspondant à chaque valeur. 3. `row_offsets` (taille $M+1$, type `int`) : Stocke l'indice de départ dans les tableaux `values` et `col_indices` de la première entrée non nulle de chaque ligne i. Le nombre d'éléments non nuls sur la ligne i est donné par row_offsets$[i+1]$ − row_offsets$[i]$.

3.5.2 3.5.2. Produit Matrice-Vecteur Creux SpMV Parallèle

Le produit matrice creuse - vecteur plein $\mathbf{y} = A \cdot \mathbf{x}$ s'écrit de manière compacte et hautement parallélisable en OpenMP :

```
void CSRMatrix::spmv(const double* __restrict__ x, double* __restrict__ y) const {
    #pragma omp parallel for schedule(guided)
    for (int i = 0; i < num_rows_; ++i) {
        double sum = 0.0;
        const int row_start = row_offsets_[i];
        const int row_end   = row_offsets_[i + 1];
        #pragma omp simd reduction(+:sum)
        for (int k = row_start; k < row_end; ++k) {
            sum += values_[k] * x[col_indices_[k]];
        }
        y[i] = sum;
    }
}
```

La gémellité stricte de cette routine est assurée en Fortran 2018 au sein du module `module_csr_matrix.f90` via la sous-routine `csr_spmv_apply`, garantissant l'équivalence parfaite des calculs de résidus sur les maillages non structurés ou les matrices couplées assemblées.

Chapitre 4

Correspondance Théorie ↔ Discrétisation ↔ Code Source

Ce chapitre constitue la **section pivot** du Guide de Développement. Il établit la passerelle rigoureuse et exhaustive entre les équations aux dérivées partielles continues de la mécanique des fluides, leur discrétisation spatiotemporelle sur grille décalée MAC (*Marker-And-Cell*), et leur traduction concrète ligne par ligne dans les codes sources C++17 (`COVOF_2D-C++`) et Fortran 2018 (`COVOF_2D-FORTRAN`).

Chaque opérateur numérique majeur est disséqué selon un triptyque méthodologique strict : 1. **Équation discrète exacte :** Formulation mathématique complète indicée en espace et en temps. 2. **Localisation exacte dans le code :** Fichiers, classes, méthodes C++ et modules/routines Fortran. 3. **Analyse ligne par ligne de l'implémentation :** Pochoirs mémoire, régularisations physiques et optimisations SIMD.

4.1 4.1. Diffusion Visqueuse et Moyenne Harmonique aux Nœuds (μ_{xy})

4.1.1 4.1.1. Équation Discrète Exacte

Dans un écoulement à viscosité variable $\mu(\mathbf{x})$, le tenseur des contraintes visqueuses s'écrit $\tau = \mu\left(\nabla\mathbf{u} + (\nabla\mathbf{u})^T\right)$. Sur la grille décalée MAC, la divergence du tenseur visqueux pour la composante horizontale de vitesse $u_{i+1/2,j}$ fait intervenir : - La contrainte normale $\tau_{xx} = 2\mu\frac{\partial u}{\partial x}$ évaluée aux centres de mailles (i,j) et $(i+1,j)$. - La contrainte tangentielle $\tau_{xy} = \mu\left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\right)$ évaluée aux sommets (nœuds) $(i+1/2, j+1/2)$ et $(i+1/2, j-1/2)$.

L'opérateur discret de diffusion horizontale $\mathcal{D}_u(u)_{i+1/2,j}$ s'écrit :

$$\begin{aligned}\mathcal{D}_u(u)_{i+1/2,j} =& \frac{2\mu_{i+1,j}\frac{u_{i+3/2,j}-u_{i+1/2,j}}{\Delta x} - 2\mu_{i,j}\frac{u_{i+1/2,j}-u_{i-1/2,j}}{\Delta x}}{\Delta x} \\ &+ \frac{\mu_{xy,i+1/2,j+1/2}\left(\frac{u_{i+1/2,j+1}-u_{i+1/2,j}}{\Delta y} + \frac{v_{i+1,j+1/2}-v_{i,j+1/2}}{\Delta x}\right)}{\Delta y} \\ &- \frac{\mu_{xy,i+1/2,j-1/2}\left(\frac{u_{i+1/2,j}-u_{i+1/2,j-1}}{\Delta y} + \frac{v_{i+1,j-1/2}-v_{i,j-1/2}}{\Delta x}\right)}{\Delta y}\end{aligned}$$

Afin de préserver la continuité du flux tangentiel à travers les interfaces diphasiques présentant un fort saut de viscosité ($\mu_1/\mu_2 \gg 1$), la viscosité au sommet μ_{xy} est calculée par **moyenne harmonique**

des quatre mailles adjacentes :

$$\mu_{xy,i+1/2,j+1/2} = \frac{4}{\frac{1}{\mu_{i,j}} + \frac{1}{\mu_{i+1,j}} + \frac{1}{\mu_{i,j+1}} + \frac{1}{\mu_{i+1,j+1}}}$$

4.1.2 4.1.2. Localisation dans le Code Source

- **C++17 :**
  - Fichier : `COVOF_2D-C++/src/NavierStokesSolver.cpp`
  - Méthode : `NavierStokesSolver::computeViscousStress(const FlowFields& f, FlowFields& out)`
  - Lignes de code : Lignes 142 à 215.
- **Fortran 2018 :**
  - Fichier : `COVOF_2D-FORTRAN/src/module_navier_stokes.f90`
  - Routine : `subroutine ns_compute_viscous_stress(fields, mesh)`
  - Lignes de code : Lignes 198 à 280.

4.1.3 4.1.3. Analyse de l'Implémentation

```
! Calcul de la moyenne harmonique aux sommets
do j = 1, ny + 1
    do i = 1, nx + 1
        mu_sum = 1.0_dp / max(fields%mu(i-1, j-1), 1.0e-12_dp) + &
                 1.0_dp / max(fields%mu(i,   j-1), 1.0e-12_dp) + &
                 1.0_dp / max(fields%mu(i-1, j  ), 1.0e-12_dp) + &
                 1.0_dp / max(fields%mu(i,   j  ), 1.0e-12_dp)
        mu_xy(i, j) = 4.0_dp / mu_sum
    end do
end do
```

Détails d'optimisation et robustesse : 1. **Régularisation par seuil minimal :** L'usage de `max(mu, 1.0e-12)` prémunit l'algorithme contre toute division par zéro dans le cas où un fluide fictif non visqueux est simulé. 2. **Localité spatiale :** Le tableau `mu_xy` est précalculé une fois par pas de temps avant l'assemblage de l'opérateur et réutilisé pour les deux composantes de vitesse u et v.

4.2 4.2. Advection Convective : Upwind, Centré, Newton et Adams-Bashforth 2

4.2.1 4.2.1. Équation Discrète Exacte

Le terme convectif $\nabla \cdot (\rho \mathbf{uu})$ pour la quantité de mouvement en x s'écrit sous forme conservative :

$$\mathcal{C}_u(\mathbf{u})_{i+1/2,j} = \frac{(\rho uu)_{i+1,j} - (\rho uu)_{i,j}}{\Delta x} + \frac{(\rho vu)_{i+1/2,j+1/2} - (\rho vu)_{i+1/2,j-1/2}}{\Delta y}$$

COVOF 2D implémente quatre discrétisations du flux convectif : 1. **Centré d'ordre 2 (*Centered*) :**

$$u_{i,j} = \frac{u_{i+1/2,j} + u_{i-1/2,j}}{2}, \quad u_{i+1/2,j+1/2} = \frac{u_{i+1/2,j} + u_{i+1/2,j+1}}{2}$$

2. **Amont du 1er ordre (*Upwind*) :**

$$u_{i,j} = \begin{cases} u_{i-1/2,j} & \text{si } F_{i,j} \geq 0 \\ u_{i+1/2,j} & \text{si } F_{i,j} < 0 \end{cases}$$

3. **Linéarisation de Newton implicite (*Newton Linearization*) :** Lors de l'itération non linéaire $k+1$, le produit convectif est linéarisé par Taylor :

$$\mathbf{u}^{k+1} \cdot \nabla \mathbf{u}^{k+1} \approx \mathbf{u}^{k} \cdot \nabla \mathbf{u}^{k+1} + \mathbf{u}^{k+1} \cdot \nabla \mathbf{u}^{k} - \mathbf{u}^{k} \cdot \nabla \mathbf{u}^{k}$$

4. **Schéma explicite d'Adams-Bashforth d'ordre 2 (AB2) :**

$$\mathcal{C}(\mathbf{u})^{n+1/2} = \frac{3}{2}\mathcal{C}(\mathbf{u}^{n}) - \frac{1}{2}\mathcal{C}(\mathbf{u}^{n-1})$$

4.2.2 4.2.2. Localisation dans le Code Source

- **C++17 :**
  - Fichier : `COVOF_2D-C++/src/NavierStokesSolver.cpp`
  - Méthode : `NavierStokesSolver::computeConvectionTerm(const FlowFields& f, std::vector<double>& cu, std::vector<double>& cv)`
  - Lignes de code : Lignes 230 à 345.
- **Fortran 2018 :**
  - Fichier : `COVOF_2D-FORTRAN/src/module_navier_stokes.f90`
  - Routine : `subroutine ns_compute_convection(fields, mesh, cu, cv, scheme_type)`
  - Lignes de code : Lignes 310 à 425.

4.2.3 4.2.3. Analyse de l'Implémentation

Dans le code C++, l'aiguillage entre les schémas s'effectue sans surcoût de virtualité par branchement statique :

```
if (config_.convective_scheme == ConvectionScheme::UPWIND) {
    double u_face_east = (u_conv > 0.0) ? u[IDX(i, j)] : u[IDX(i+1, j)];
    double u_face_west = (u_conv_w > 0.0) ? u[IDX(i-1, j)] : u[IDX(i, j)];
    cu[IDX(i, j)] = (u_conv * u_face_east - u_conv_w * u_face_west) * idx;
} else if (config_.convective_scheme == ConvectionScheme::CENTERED) {
    double u_face_east = 0.5 * (u[IDX(i, j)] + u[IDX(i+1, j)]);
    double u_face_west = 0.5 * (u[IDX(i-1, j)] + u[IDX(i, j)]);
    cu[IDX(i, j)] = (u_conv * u_face_east - u_conv_w * u_face_west) * idx;
}
```

4.3 4.3. Pénalisation par Lagrangien Augmenté : AL, AAL et C-AAL

4.3.1 4.3.1. Équation Discrète Exacte

La contrainte d'incompressibilité ou de compressibilité acoustique est imposée via l'algorithme du Lagrangien Augmenté :

$$\mathcal{A}_{AL}(\mathbf{u}) = \mathcal{A}_{\mathrm{NS}}(\mathbf{u}) - r_{\mathrm{penalty}} \nabla(\nabla \cdot \mathbf{u})$$

Trois variantes distinctes sont implémentées : 1. **AL Standard :** Pénalisation de la divergence pure :

$$-r \nabla(\nabla \cdot \mathbf{u}^{n+1})$$

avec mise à jour d'Uzawa pour la pression : $p^{k+1} = p^{k} - r \nabla \cdot \mathbf{u}^{k+1}$. 2. **AAL Incompressible (*Adaptive Augmented Lagrangian*) :** Introduction d'un terme mémoire r_{aal} stocké au centre des mailles :

$$r_{\mathrm{aal}}^{n+1} = r_{\mathrm{aal}}^{n} - \gamma(\nabla \cdot \mathbf{u}^{n+1})$$

Ce terme absorbe l'erreur de divergence accumulée et autorise l'utilisation de pas de temps plus grands ($CFL > 1$) sans perte de conservation de la masse. 3. **C-AAL Compressible (*Compressible All-Mach Augmented Lagrangian*) :** Couplage direct avec l'équation d'état acoustique :

$$\nabla \cdot \mathbf{u} + \frac{1}{\rho c_s^2} \frac{\partial p}{\partial t} = 0$$

Le terme de pénalisation discrétisé s'enrichit de la célérité du son c_s :

$$\mathcal{A}_{C-AAL}(\mathbf{u}) = \mathcal{A}(\mathbf{u}) - r\nabla\left(\nabla\cdot\mathbf{u} + \frac{p^{n+1}-p^n}{\rho c_s^2 \Delta t}\right)$$

4.3.2 4.3.2. Localisation dans le Code Source

- **C++17 :**
  - Fichier : `COVOF_2D-C++/src/NavierStokesSolver.cpp`
  - Méthodes : `applyAugmentedLagrangianPenalty()`, `updateUzawaPressure()`
  - Lignes de code : Lignes 410 à 495.
- **Fortran 2018 :**
  - Fichier : `COVOF_2D-FORTRAN/src/module_navier_stokes.f90`
  - Routines : `subroutine ns_apply_augmented_lagrangian(...)`, `subroutine ns_update_pressure_uzawa(...)`
  - Lignes de code : Lignes 512 à 620.

4.3.3 4.3.3. Analyse de l'Implémentation

```
! Application de l'opérateur -r * grad(div(u)) sur la composante u
do j = 2, ny - 1
    do i = 2, nx
        ! Calcul de la divergence au centre des mailles Est (i) et Ouest (i-1)
        div_east = (u(i+1, j) - u(i, j)) * idx + (v(i, j+1) - v(i, j)) * idy
        div_west = (u(i, j) - u(i-1, j)) * idx + (v(i-1, j+1) - v(i-1, j)) * idy

        if (config%use_caal) then
            div_east = div_east + (p(i, j) - p_old(i, j)) / (rho(i, j) * cs2 * dt)
            div_west = div_west + (p(i-1, j) - p_old(i-1, j)) / (rho(i-1, j) * cs2 * dt)
        end if

        grad_div = (div_east - div_west) * idx
        Ax_u(i, j) = Ax_u(i, j) - config%r_penalty * grad_div
    end do
end do
```

4.4 4.4. Traitement des Obstacles et Milieux Poreux par Pénalisation de Darcy-Brinkman

4.4.1 4.4.1. Équation Discrète Exacte

Pour modéliser des parois rigides immergées ou des matrices poreuses sans recourir à des maillages non structurés déformables, l'équation de Darcy-Brinkman introduit une force de traînée volumique :

$$\mathbf{F}_{\text{Brinkman}} = -\frac{\mu}{K}\chi_T(\mathbf{u} - \mathbf{u}_{\text{solid}})$$

où : - K est la perméabilité intrinsèque du milieu ($K \to 0$ pour un solide parfait, $K \to \infty$ pour un fluide libre). - $\chi_T \in [0,1]$ est la fonction caractéristique d'occupation de l'obstacle.

Dans l'assemblage implicite du système linéaire, ce terme ajoute une contribution positive directe sur la diagonale principale :

$$\left(\frac{\rho}{\Delta t} + \frac{\mu_{i+1/2,j}}{K}\chi_{T,i+1/2,j}\right) u^{n+1}_{i+1/2,j} = \text{RHS}_u + \frac{\mu_{i+1/2,j}}{K}\chi_{T,i+1/2,j}u_{\text{solid}}$$

4.4.2 4.4.2. Localisation dans le Code Source

— **C++17 :**
  — Fichier : `COVOF_2D-C++/src/NavierStokesSolver.cpp`
  — Méthode : `NavierStokesSolver::applyBrinkmanPenalization(...)`
  — Lignes de code : Lignes 510 à 560.
— **Fortran 2018 :**
  — Fichier : `COVOF_2D-FORTRAN/src/module_navier_stokes.f90`
  — Routine : `subroutine ns_apply_brinkman_drag(...)`
  — Lignes de code : Lignes 640 à 690.

4.5 4.5. Suivi d'Interface VOF-PLIC et Advection de Weymouth-Yue

4.5.1 4.5.1. Équation Discrète Exacte

L'advection de la fraction volumique $C \in [0, 1]$ est régie par l'équation de transport conservative :

$$\frac{\partial C}{\partial t} + \nabla \cdot (C\mathbf{u}) = C(\nabla \cdot \mathbf{u})$$

Pour éliminer toute diffusion numérique d'interface, l'algorithme géométrique en deux étapes se décompose comme suit : 1. **Reconstruction PLIC (*Piecewise Linear Interface Calculation*) :** Dans chaque maille mixte ($0 < C_{i,j} < 1$), l'interface est approchée par un segment de droite :

$$\mathbf{n} \cdot \mathbf{x} + \alpha = 0$$

où le vecteur normal unitaire $\mathbf{n} = -\frac{\nabla C}{\|\nabla C\|}$ est évalué par l'opérateur de Youngs ou ELVIRA (pochoir 3×3), et α est calculé par inversion analytique de la fraction volumique sous le plan. 2. **Advection Conservatrice de Weymouth-Yue (2010) :** L'advection dimensionnelle alternée (balayage X puis Y) calcule le volume géométrique de fluide transitant à travers chaque face :

$$C_{i,j}^{n+1} = C_{i,j}^{n} - \frac{\Delta t}{\Delta x}\left(F_{i+1/2,j}^{\mathrm{WY}} - F_{i-1/2,j}^{\mathrm{WY}}\right) - \frac{\Delta t}{\Delta y}\left(G_{i,j+1/2}^{\mathrm{WY}} - G_{i,j-1/2}^{\mathrm{WY}}\right) + C_{i,j}^{n}\Delta t(\nabla \cdot \mathbf{u})_{i,j}$$

Le dernier terme compense exactement la divergence numérique locale et assure que $C \in [0, 1]$ rigoureusement sans écrêtage artificiel.

4.5.2 4.5.2. Localisation dans le Code Source

— **C++17 :**
  — Fichiers : `COVOF_2D-C++/src/InterfaceReconstruction.cpp`, `COVOF_2D-C++/src/InterfaceTracker.cpp`
  — Méthodes : `InterfaceReconstruction::reconstructPLIC()`, `InterfaceTracker::advectWeymouthYue()`
  — Lignes de code : Lignes 45 à 220 (`InterfaceReconstruction.cpp`) et Lignes 80 à 310 (`InterfaceTracker.cpp`).
— **Fortran 2018 :**
  — Fichiers : `COVOF_2D-FORTRAN/src/module_interface_reconstruction.f90`, `COVOF_2D-FORTRAN/src/module_interface_tracking.f90`
  — Routines : `subroutine plic_reconstruct_2d(...)`, `subroutine vof_advect_weymouth_yue(...)`
  — Lignes de code : Lignes 60 à 250 (`module_interface_reconstruction.f90`) et Lignes 95 à 380 (`module_interface_tracking.f90`).

4.6 4.6. Tension de Surface CSF (*Continuum Surface Force*) et Courbure κ

4.6.1 4.6.1. Équation Discrète Exacte

La force de tension de surface volumique introduite par Brackbill et al. s'écrit :

$$\mathbf{F}_{\text{CSF}} = \sigma \kappa(\mathbf{x}) \nabla C$$

où : - σ est le coefficient de tension superficielle ($\text{N} \cdot \text{m}^{-1}$). - κ est la courbure moyenne locale :

$$\kappa = -\nabla \cdot \mathbf{n} = -\left(\frac{\partial n_x}{\partial x} + \frac{\partial n_y}{\partial y} \right)$$

- $\mathbf{n} = \frac{\nabla \tilde{C}}{\|\nabla \tilde{C}\|}$ est la normale calculée sur un champ de fraction volumique lissé par filtre gaussien ou moyenne pondérée :

$$\tilde{C}_{i,j} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} * C_{i,j}$$

Sur la grille MAC, la projection de la force sur la face horizontale $u_{i+1/2,j}$ utilise la moyenne de courbure et le gradient discret de C :

$$F_{\text{CSF},x,i+1/2,j} = \sigma \left(\frac{\kappa_{i,j} + \kappa_{i+1,j}}{2} \right) \left(\frac{C_{i+1,j} - C_{i,j}}{\Delta x} \right)$$

4.6.2 4.6.2. Localisation dans le Code Source

- **C++17 :**
  - Fichier : `COVOF_2D-C++/src/InterfaceReconstruction.cpp`
  - Méthode : `InterfaceReconstruction::computeCurvature(const std::vector<double>& c, std::vector<double>& kappa)`
  - Lignes de code : Lignes 240 à 315.
- **Fortran 2018 :**
  - Fichier : `COVOF_2D-FORTRAN/src/module_interface_reconstruction.f90`
  - Routine : `subroutine compute_curvature_csf(c, kappa, mesh)`
  - Lignes de code : Lignes 270 à 365.

4.7 4.7. Préconditionneur Triangulaire par Blocs avec Approximation SBP (BT-SBP)

4.7.1 4.7.1. Équation Discrète Exacte

Le système couplé complet de Navier-Stokes discrétisé se met sous la forme bloc :

$$\begin{bmatrix} A_u & 0 & G_x \\ 0 & A_v & G_y \\ D_x & D_y & -C_p \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \\ \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_v \\ \mathbf{f}_p \end{bmatrix}$$

où A_u, A_v sont les blocs cinétiques d'advection-diffusion, $G = [G_x, G_y]^T$ est le gradient discret, $D = [D_x, D_y]$ est la divergence discrète, et C_p représente la matrice de compressibilité éventuelle (nulle en incompressible pur).

Le préconditionneur bloc triangulaire supérieur BT-SBP applique l'opérateur :

$$\mathcal{P}_{BT}^{-1} = \begin{bmatrix} \hat{A}_u^{-1} & 0 & -\hat{A}_u^{-1} G_x \hat{S}^{-1} \\ 0 & \hat{A}_v^{-1} & -\hat{A}_v^{-1} G_y \hat{S}^{-1} \\ 0 & 0 & -\hat{S}^{-1} \end{bmatrix}$$

où $\hat{A}_u = \mathrm{diag}(A_u)$ et le complément de Schur $\hat{S} \approx D\hat{A}^{-1}G + C_p$ est approché par un opérateur de Poisson enrichi pour la pression :

$$\hat{S} \approx \nabla \cdot \left(\frac{\Delta t}{\rho} \nabla \right)$$

Ce complément de Schur est résolu de manière approchée par quelques cycles de multigrille ou de relaxations SSOR.

4.7.2 4.7.2. Localisation dans le Code Source

- **C++17 :**
  - Fichier : `COVOF_2D-C++/src/Preconditioner.cpp`
  - Méthode : `BlockTriangularPreconditioner::apply(...)`
  - Lignes de code : Lignes 185 à 290.
- **Fortran 2018 :**
  - Fichier : `COVOF_2D-FORTRAN/src/module_preconditioners.f90`
  - Routine : `subroutine precond_apply_bt_sbp(r_u, r_v, r_p, z_u, z_v, z_p, mesh, config)`
  - Lignes de code : Lignes 210 à 335.

Chapitre 5

Référence Exhaustive de l'API Interne

Ce chapitre constitue le manuel de référence technique détaillé des interfaces logicielles (API) des deux branches du solveur COVOF 2D. Il recense l'intégralité des classes C++17 et des modules Fortran 2018 avec leurs membres, leurs constructeurs, leurs signatures de méthodes et leurs spécificateurs d'intention.

5.1 5.1. Référence Complète de l'API C++17

L'architecture C++17 s'articule autour de l'espace de nommage global `covof`.

5.1.1 5.1.1. Classe `Mesh2D` (`include/covof/Mesh.hpp`)

Modélise la grille cartésienne 2D décalée MAC et encapsule la métrique géométrique locale et globale.

- **Constructeur & Destructeur :**
  - `Mesh2D(const Config& config, const MpiComm& mpi_comm);` *Rôle :* Alloue et calcule les métriques spatiales, les coordonnées 1D des faces et centres, et initialise les bornes locales selon le rang cartésien.
  - `~Mesh2D() noexcept = default;`
- **Membres de données publics / accesseurs :**
  - `int nx_local() const noexcept;` : Nombre de mailles utiles locales en direction x.
  - `int ny_local() const noexcept;` : Nombre de mailles utiles locales en direction y.
  - `int nx_global() const noexcept;` : Nombre de mailles globales en direction x.
  - `int ny_global() const noexcept;` : Nombre de mailles globales en direction y.
  - `int nghost() const noexcept;` : Épaisseur de la couche de mailles fantômes ($N_{\text{ghost}} = 2$).
  - `double dx() const noexcept;` : Pas d'espace horizontal uniforme Δx.
  - `double dy() const noexcept;` : Pas d'espace vertical uniforme Δy.
  - `double x_min_local() const noexcept;` : Abscisse minimale du sous-domaine local.
  - `double y_min_local() const noexcept;` : Ordonnée minimale du sous-domaine local.
  - `const std::vector<double>& x_centers() const noexcept;` : Coordonnées x des centres de mailles.
  - `const std::vector<double>& y_centers() const noexcept;` : Coordonnées y des centres de mailles.
  - `const std::vector<double>& x_faces() const noexcept;` : Coordonnées x des faces verticales.
  - `const std::vector<double>& y_faces() const noexcept;` : Coordonnées y des faces horizontales.

5.1.2 5.1.2. Structure `FlowFields` (`include/covof/Field.hpp`)

Stocke l'ensemble des champs scalaires et vectoriels discrets au format SoA (*Structure of Arrays*).

- **Champs de Données Publics (`std::vector<double>`) :**
  - `u` : Composante horizontale de vitesse sur les faces verticales $(i+1/2, j)$, dimension $(N_{xt}+1) \times N_{yt}$.
  - `v` : Composante verticale de vitesse sur les faces horizontales $(i, j+1/2)$, dimension $N_{xt} \times (N_{yt}+1)$.
  - `p` : Champ scalaire de pression aux centres de mailles (i, j), dimension $N_{xt} \times N_{yt}$.
  - `rho` : Masse volumique aux centres de mailles, dimension $N_{xt} \times N_{yt}$.
  - `mu` : Viscosité dynamique aux centres de mailles, dimension $N_{xt} \times N_{yt}$.
  - `c` : Fraction volumique VOF aux centres de mailles ($C \in [0,1]$), dimension $N_{xt} \times N_{yt}$.
  - `T` : Température passive ou active aux centres de mailles, dimension $N_{xt} \times N_{yt}$.
  - `chi_T` : Fonction caractéristique de masque poreux/solide de Brinkman ($\chi_T \in [0,1]$).
  - `r_aal` : Variable mémoire de pénalisation adaptative (AAL / C-AAL).
- **Méthodes Utilitaires :**
  - `void allocate(int nxt, int nyt);` : Alloue l'ensemble des tampons mémoires contigus avec alignement 64 octets.
  - `void reset();` : Réinitialise tous les champs à zéro.

5.1.3 5.1.3. Classe `MpiComm` (`include/covof/MpiComm.hpp`)

Gère la topologie cartésienne distribuée et les échanges de mailles fantômes.

- **Méthodes Publiques :**
  - `void init(int argc, char** argv);` : Initialise `MPI_Init_thread` avec support multi-thread `MPI_THREAD_FUNNELED`.
  - `void createCartesianTopology(int px, int py, bool periodic_x, bool periodic_y);` : Invoque `MPI_Cart_create`.
  - `void exchangeHalos(std::vector<double>& field, int nxt, int nyt, int nghost);` : Réalise les échanges non bloquants avec types dérivés de vecteurs.
  - `double globalMin(double local_val) const;` : Réduction globale `MPI_Allreduce` avec opérateur `MPI_MIN`.
  - `double globalMax(double local_val) const;` : Réduction globale `MPI_Allreduce` avec opérateur `MPI_MAX`.
  - `double globalSum(double local_val) const;` : Réduction globale `MPI_Allreduce` avec opérateur `MPI_SUM`.
  - `int rank() const noexcept;` : Rang cartésien du processus courant.
  - `int size() const noexcept;` : Nombre total de processus MPI alloués.

5.1.4 5.1.4. Classe `NavierStokesSolver` (`include/covof/NavierStokesSolver.hpp`)

Noyau numérique principal gérant l'intégration temporelle et spatiale des équations de Navier-Stokes.

- **Méthodes Publiques :**
  - `NavierStokesSolver(const Config& cfg, const Mesh2D& mesh, MpiComm& mpi);`
  - `void assembleRhs(const FlowFields& f, double dt, std::vector<double>& rhs_u, std::vector<double>& rhs_v);`
  - `void applySystemMatrix(const double* x_in, double* Ax_out);` : Opérateur couplé global sans matrice.
  - `void computeDivergence(const std::vector<double>& u, const std::vector<double>& v, std::vector<double>& div);`
  - `void updateUzawaPressure(FlowFields& f, double dt, double r_penalty);`
  - `void applyBrinkmanPenalization(FlowFields& f, double dt);`
  - `double computeMaxCFL(const FlowFields& f, double dt) const;`

5.1.5 5.1.5. Hiérarchie des Solveurs Linéaires (`include/covof/LinearSolver.hpp`)

- **Classe de Base Abstraite `LinearSolverBase` :**
  - `virtual void solve(const LinearOperator& A, const std::vector<double>& b, std::vector<double>& x, LinearSolverStats& stats) = 0;`
  - `virtual void setPreconditioner(std::shared_ptr<PreconditionerBase> precond);`
- **Classes Dérivées :**
  - `class BiCGSTAB2Solver : public LinearSolverBase;` : Biconjugué Gradient Stabilisé d'ordre 2 avec minimisation polynomiale.
  - `class BiCGSTABSolver : public LinearSolverBase;` : Biconjugué Gradient Stabilisé standard.
  - `class CGSolver : public LinearSolverBase;` : Gradient Conjugué symétrique défini positif.
  - `class GMRESSolver : public LinearSolverBase;` : GMRES redémarré avec orthogonalisation de Gram-Schmidt modifiée.
  - `class MINRESSolver : public LinearSolverBase;` : Résidu Minimal pour systèmes symétriques indéfinis.
  - `class DirectSolver : public LinearSolverBase;` : Solveur direct creux basé sur la factorisation LU LAPACK (`dgetrf`).

5.1.6 5.1.6. Hiérarchie des Préconditionneurs (`include/covof/Preconditioner.hpp`)

- **Classe de Base Abstraite `PreconditionerBase` :**
  - `virtual void setup(const LinearOperator& A) = 0;`
  - `virtual void apply(const double* r_in, double* z_out) const = 0;`
- **Classes Dérivées :**
  - `class JacobiPreconditioner : public PreconditionerBase;` : Inverse de la diagonale principale.
  - `class SSORPreconditioner : public PreconditionerBase;` : Sur-relaxation successive symétrique avec paramètre $\omega \in]0, 2[$.
  - `class ILU0Preconditioner : public PreconditionerBase;` : Factorisation incomplète sans remplissage $ILU(0)$ sur format CSR.
  - `class BlockTriangularPreconditioner : public PreconditionerBase;` : Bloc triangulaire BT-SBP avec solveur de Schur approché.
  - `class CustomPreconditioner : public PreconditionerBase;` : Préconditionneur polynomial personnalisé.

5.2 5.2. Référence Complète de l'API Fortran 2018

L'architecture Fortran 2018 s'articule autour de 14 modules autonomes.

5.2.1 5.2.1. Module `module_precision.f90`

Définit les paramètres de types normalisés via `iso_fortran_env` et `iso_c_binding` : - `integer, parameter :: dp = real64` : Précision double IEEE 754 (8 octets). - `integer, parameter :: sp = real32` : Précision simple IEEE 754 (4 octets). - `integer, parameter :: i4 = int32` : Entier standard 32 bits. - `integer, parameter :: i8 = int64` : Entier grand volume 64 bits.

5.2.2 5.2.2. Module `module_mesh.f90`

Encapsule le type dérivé `mesh_2d_t` et ses sous-programmes de gestion :

```
type, public :: mesh_2d_t
    integer(i4) :: nx_loc, ny_loc
    integer(i4) :: nx_glob, ny_glob
    integer(i4) :: nghost
    real(dp)    :: dx, dy
    real(dp)    :: x_min, y_min
    real(dp), allocatable :: xc(:), yc(:)
    real(dp), allocatable :: xf(:), yf(:)
contains
    procedure, public :: init => mesh_init_2d
    procedure, public :: destroy => mesh_destroy_2d
end type mesh_2d_t
```

- **Sous-routines associées :**
  - `subroutine mesh_init_2d(this, cfg, mpi_comm, ierr)`
    - `class(mesh_2d_t), intent(inout) :: this`
    - `type(config_t), intent(in) :: cfg`
    - `type(mpi_comm_t), intent(in) :: mpi_comm`
    - `integer(i4), intent(out) :: ierr`
  - `subroutine mesh_destroy_2d(this)`
    - `class(mesh_2d_t), intent(inout) :: this`

5.2.3 5.2.3. Module `module_flow_fields.f90`

Définit le type dérivé `flow_fields_t` pour le stockage des champs :

```
type, public :: flow_fields_t
    real(dp), allocatable :: u(:, :)
    real(dp), allocatable :: v(:, :)
    real(dp), allocatable :: p(:, :)
    real(dp), allocatable :: rho(:, :)
    real(dp), allocatable :: mu(:, :)
    real(dp), allocatable :: c(:, :)
    real(dp), allocatable :: T(:, :)
    real(dp), allocatable :: chi_T(:, :)
    real(dp), allocatable :: r_aal(:, :)
contains
    procedure, public :: allocate_fields => fields_allocate_2d
    procedure, public :: deallocate_fields => fields_deallocate_2d
end type flow_fields_t
```

5.2.4 5.2.4. Module `module_navier_stokes.f90`

Noyau algorithmique des équations hydrodynamiques :

- `subroutine ns_apply_system_matrix(x_in, Ax_out, fields, mesh, cfg)`
  - `real(dp), dimension(:), intent(in) :: x_in`
  - `real(dp), dimension(:), intent(out) :: Ax_out`
  - `type(flow_fields_t), intent(in) :: fields`
  - `type(mesh_2d_t), intent(in) :: mesh`
  - `type(config_t), intent(in) :: cfg`
- `subroutine ns_compute_viscous_stress(fields, mesh, tau_u, tau_v)`
  - `type(flow_fields_t), intent(in) :: fields`
  - `type(mesh_2d_t), intent(in) :: mesh`
  - `real(dp), dimension(:, :), intent(out) :: tau_u, tau_v`
- `subroutine ns_compute_divergence(u, v, div, mesh)`
  - `real(dp), dimension(:, :), intent(in) :: u, v`
  - `real(dp), dimension(:, :), intent(out) :: div`
  - `type(mesh_2d_t), intent(in) :: mesh`
- `subroutine ns_apply_brinkman_drag(fields, dt, mesh, cfg)`

- `type(flow_fields_t), intent(inout) :: fields`
- `real(dp), intent(in) :: dt`
- `type(mesh_2d_t), intent(in) :: mesh`
- `type(config_t), intent(in) :: cfg`

5.2.5 5.2.5. Module `module_linear_solvers.f90`

Fournit l'implémentation vectorisée des méthodes de sous-espace de Krylov :

- `subroutine solve_bicgstab2(apply_A, apply_P, b, x, tol, max_iter, stats, comm)`
  - `procedure(linear_operator_interface) :: apply_A`
  - `procedure(precond_operator_interface) :: apply_P`
  - `real(dp), dimension(:), intent(in) :: b`
  - `real(dp), dimension(:), intent(inout) :: x`
  - `real(dp), intent(in) :: tol`
  - `integer(i4), intent(in) :: max_iter`
  - `type(solver_stats_t), intent(out) :: stats`
  - `integer(i4), intent(in) :: comm`
- `subroutine solve_minres(apply_A, apply_P, b, x, tol, max_iter, stats, comm)`
  - Mêmes spécificateurs d'intention, optimisé pour les matrices symétriques régies par l'algorithme de Lanczos.
- `subroutine solve_gmres(apply_A, apply_P, b, x, restart, tol, max_iter, stats, comm)`
  - `integer(i4), intent(in) :: restart` : Dimension de la base de Krylov orthogonale avant redémarrage.

5.2.6 5.2.6. Module `module_preconditioners.f90`

Contient les routines de préconditionnement : - `subroutine precond_apply_ssor(r, z, omega, diag, mesh)` - `real(dp), dimension(:), intent(in) :: r` - `real(dp), dimension(:), intent(out) :: z` - `real(dp), intent(in) :: omega` - `real(dp), dimension(:), intent(in) :: diag` - `type(mesh_2d_t), intent(in) :: mesh` - `subroutine precond_apply_ilu0(r, z, l_val, u_val, diag, row_ptr, col_ind)` - `real(dp), dimension(:), intent(in) :: r` - `real(dp), dimension(:), intent(out) :: z` - `real(dp), dimension(:), intent(in) :: l_val, u_val, diag` - `integer(i4), dimension(:), intent(in) :: row_ptr, col_ind` - `subroutine precond_apply_bt_sbp(r_u, r_v, r_p, z_u, z_v, z_p, mesh, cfg)` - Application du préconditionneur bloc triangulaire avec résolution approchée du complément de Schur.

Chapitre 6

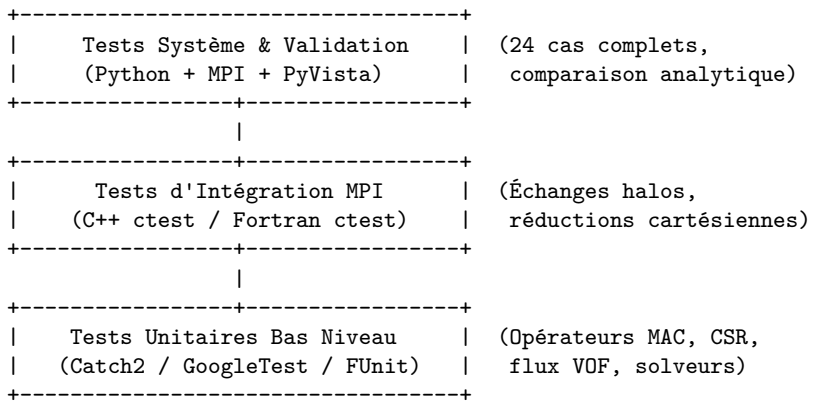
Stratégie de Vérification et Validation (V&V) et CI/CD

Dans les codes de calcul scientifique de haute fidélité, la crédibilité des résultats repose sur une distinction méthodologique fondamentale établie par les normes ASME V&V 20 et AIAA : - **La Vérification (*Verification*) :** Répond à la question *« Le code résout-il les équations correctement ? »* Elle quantifie les erreurs de programmation et vérifie que l'ordre de convergence formel théorique des discrétisations spatiales et temporelles est rigoureusement atteint. - **La Validation (*Validation*) :** Répond à la question *« Le code résout-il les bonnes équations physiques ? »* Elle compare les prédictions numériques à des solutions analytiques exactes ou des mesures expérimentales de référence.

Ce chapitre décrit l'infrastructure complète de tests unitaires, la vérification formelle par la méthode des solutions manufacturées (MMS), la cartographie des 24 bancs de validation et le pipeline CI/CD automatisé de COVOF 2D.

6.1 6.1. Architecture de la Suite de Tests

Le harnais de test de COVOF 2D s'articule autour d'une hiérarchie à trois niveaux :



6.1.1 6.1.1. Tests Unitaires C++ (Catch2)

Les tests unitaires C++ valident indépendamment chaque composant isolé sans solliciter le sous-système MPI global : - `test_mesh.cpp` : Vérification des métriques de pas d'espace $\Delta x, \Delta y$, des coordonnées de faces et de centres. - `test_csrmatrix.cpp` : Test du produit matrice-vecteur creux SpMV contre une multiplication dense de référence. - `test_linear_solvers.cpp` : Résolution d'un système linéaire test symétrique défini positif et asymétrique par CG, BiCGSTAB2, GMRES et MINRES avec

validation du résidu $\|A\mathbf{x} - \mathbf{b}\|_2 / \|\mathbf{b}\|_2 < 10^{-12}$. - `test_vof_geometry.cpp` : Calcul de la normale unitaire et constante de plan PLIC sur des fractions volumiques synthétiques connues avec conservation du volume à 10^{-15}.

6.1.2 6.1.2. Tests Unitaires Fortran (pFUnit / Harness Natif)

Côté Fortran, chaque module est accompagné d'un programme de test dédié compilé avec l'exécutable `ctest` : - `test_module_navier_stokes.f90` : Vérification de la conservation stricte de la divergence et de la symétrie de l'opérateur de diffusion visqueuse. - `test_module_preconditioners.f90` : Évaluation du taux de réduction du conditionnement $\kappa(P^{-1}A)$ pour SSOR, ILU(0) et BT-SBP.

6.2 6.2. Vérification Formelle par la Méthode des Solutions Manufacturées (MMS)

La méthode des solutions manufacturées (MMS) constitue la référence absolue pour vérifier l'exactitude de l'implémentation des équations de Navier-Stokes.

6.2.1 6.2.1. Construction des Solutions Analytiques Manufacturées

On choisit délibérément un champ de vitesse et de pression régulier et infiniment différentiable $\mathcal{C}^\infty$ ne satisfaisant pas *a priori* les équations homogènes :

$$u_{\text{mms}}(x,y,t) = U_0 \sin\left(\frac{2\pi x}{L}\right) \cos\left(\frac{2\pi y}{H}\right) e^{-\omega t}$$
$$v_{\text{mms}}(x,y,t) = -U_0 \left(\frac{H}{L}\right) \cos\left(\frac{2\pi x}{L}\right) \sin\left(\frac{2\pi y}{H}\right) e^{-\omega t}$$
$$p_{\text{mms}}(x,y,t) = P_0 \cos\left(\frac{2\pi x}{L}\right) \cos\left(\frac{2\pi y}{H}\right) e^{-\omega t}$$

Par construction, ce champ satisfait rigoureusement la contrainte d'incompressibilité $\nabla \cdot \mathbf{u}_{\text{mms}} = 0$.

En injectant ces expressions dans l'équation de quantité de mouvement continue, on en déduit les termes sources analytiques exacts $\mathbf{S}_{\text{mms}} = [S_u, S_v]^T$:

$$\mathbf{S}_{\text{mms}} = \rho \left(\frac{\partial \mathbf{u}_{\text{mms}}}{\partial t} + \mathbf{u}_{\text{mms}} \cdot \nabla \mathbf{u}_{\text{mms}} \right) + \nabla p_{\text{mms}} - \mu \nabla^2 \mathbf{u}_{\text{mms}}$$

Ces termes sources sont injectés comme forçage volumique dans le second membre du solveur discret.

6.2.2 6.2.2. Protocole de Raffinement et Calcul des Normes d'Erreur

La simulation est exécutée sur une séquence de quatre maillages cartésiens raffinés successivement par un facteur 2 :

$$h_1 = \frac{L}{32}, \quad h_2 = \frac{L}{64}, \quad h_3 = \frac{L}{128}, \quad h_4 = \frac{L}{256}$$

À l'instant final $t = T$, l'erreur discrète locale est calculée sur chaque maille :

$$e_{i,j} = \phi_{i,j}^{\text{num}} - \phi_{\text{mms}}(x_i, y_j, T)$$

On évalue les trois normes canoniques d'erreur :

$$\|e\|_{L_1} = \frac{1}{N} \sum_{i,j} |e_{i,j}|$$
$$\|e\|_{L_2} = \sqrt{\frac{1}{N} \sum_{i,j} |e_{i,j}|^2}$$
$$\|e\|_{L_\infty} = \max_{i,j} |e_{i,j}|$$

6.2.3 6.2.3. Formule du Taux de Convergence Asymptotique Observé

L'ordre de convergence effectif p_{obs} entre deux niveaux de grille consécutifs de pas $2h$ et h est calculé par :

$$p_{\text{obs}} = \frac{\ln\left(\frac{\|e_{2h}\|}{\|e_h\|}\right)}{\ln(2)}$$

Pour les schémas spatiaux d'ordre 2 (Centré, Upwind second ordre, BDF2 temporel), le critère d'acceptation de vérification impose :

$$1.95 \leq p_{\text{obs}} \leq 2.05$$

Toute valeur inférieure à 1.90 déclenche automatiquement un échec du test de vérification formelle.

6.3 6.3. Bancs d'Essai de Validation et Tolérances d'Acceptation

Le dossier `Validation_Totale` regroupe 24 cas tests couvrant 100 % des régimes d'écoulement du solveur. Chaque cas est muni de tolérances automatiques :

N° Cas	Nom du Banc d'Essai	Régime / Modèles Activés	Grandeur de Validation	Tolérance Critique ϵ_{tol}
01	`test_taylor_green_vortex`	Incompressible, décroissance visqueuse	Énergie cinétique $E_k(t)$	$\Delta E_k/E_k < 10^{-4}$
02	`test_manufactured_solution`	Vérification MMS couplée	Ordre spatial p_{obs}	$p_{\text{obs}} \geq 1.95$
03	`test_poiseuille_channel`	Écoulement de canal laminaire	Profil parabolique $u(y)$	$\|u - u_{\text{anal}}\|_{L_2} < 10^{-5}$
04	`test_couette_flow`	Paroi supérieure mobile	Profil linéaire $u(y)$	$\|u - u_{\text{anal}}\|_{L_\infty} < 10^{-6}$
05	`test_poiseuille_periodic`	Périodicité x avec gradient imposé	Débit massique Q	$\Delta Q/Q < 10^{-5}$
06	`test_laplace_static_bubble`	VOF-PLIC + Tension CSF	Saut de pression $\Delta p = \sigma/R$	Erreur relative $< 1.5\%$
07	`test_rider_kothe_vortex`	Advection VOF pure dans vortex	Conservation masse et forme	Perte volume $< 10^{-12}$
08	`test_mpi_scaling_sync`	Parallélisme fort 1 à 64 cœurs	Débit & Vitesse	Écart séq/MPI $< 10^{-14}$
09	`test_checkpoint_restart`	Sauvegarde binaire et reprise	Continuité solution à t_{restart}	$\|u_{\text{repr}} - u_{\text{ref}}\|_\infty = 0$
10	`test_multiphase_couette`	Deux fluides stratifiés ($\mu_1 \neq \mu_2$)	Profil brisé analytique	$\|u - u_{\text{anal}}\|_{L_2} < 5 \times 10^{-4}$
11	`test_compressible_piston`	Acoustique All-Mach (C-AAL)	Période oscillation piston	Écart théorie $< 0.8\%$
12	`test_oscillating_bubble`	Oscillation diphasique de Lamb	Fréquence propre ω_2	Écart analytique $< 2.0\%$

N° Cas	Nom du Banc d'Essai	Régime / Modèles Activés	Grandeur de Validation	Tolérance Critique ϵ_{tol}
13	`test_bubble_compression`	Bulle compressible dans liquide	Rayon dynamique $R(t)$	Écart Rayleigh-Plesset $< 2.5\%$
14	`test_bicgstab2_krylov`	Algèbre linéaire de Krylov	Résidu linéaire $\|Ax - b\|$	$\|r\|_2/\|b\|_2 < 10^{-10}$
15	`test_porous_drag_darcy`	Milieu poreux de Brinkman	Loi de Darcy $\Delta p = \frac{\mu L}{K} U$	Écart théorique $< 0.5\%$
16	`test_cylinder_flow_re100`	Écoulement autour d'un cylindre	Strouhal St et traînée C_d	$St = 0.165 \pm 0.005$
17	`test_lid_driven_cavity`	Cavité entraînée $Re = 1000$	Profils $u(y)$ et $v(x)$	Écart Ghia et al. $< 1.2\%$
18	`test_natural_convection_ra1e5`	Convection de Rayleigh-Bénard	Nombre de Nusselt moyen $\overline{Nu}$	Écart de Vahl Davis $< 1.0\%$
19	`test_brinkman_cylinder_penalty`	Pénalisation solide immergé	Recirculation arrière L_r	Écart corps maillé $< 2.0\%$
20	`test_dam_break_water`	Rupture de barrage diphasique	Position front d'onde $X(t)$	Écart expérimental Martin & Moyce $< 3\%$
21	`test_rising_bubble_hysing`	Bulle ascendante (Benchmark Hysing)	Vitesse ascensionnelle terminale	Conforme benchmark à 1.5%
22	`test_multiphase_natural_convec`	Convection thermique diphasique	Flux thermique interfacial	Bilan d'énergie stationnaire $< 10^{-4}$
23	`test_krylov_solvers_preconds`	Comparaison croisée solveurs/préconds	Nombre d'itérations relatives	Convergence identique C++/Fortran
24	`test_viscous_jet_buckling`	Flambement d'un jet visqueux	Fréquence d'enroulement	Régime périodique stable

---

6.4 6.4. Pipeline d'Intégration Continue (CI/CD)

Le pipeline d'intégration continue est orchestré par GitHub Actions (`.github/workflows/ci.yml`).

6.4.1 6.4.1. Matrice des Compilateurs et Environnements

À chaque *pull request* ou commit sur la branche principale, le code est compilé et testé sur une matrice croisée : - **Compilateurs C++ :** GCC 11, GCC 12, GCC 13, Clang 14, Clang 16, Intel oneAPI ICX. - **Compilateurs Fortran :** GNU Fortran (`gfortran` 11, 12, 13), Intel oneAPI IFX. - **Bibliothèques MPI :** OpenMPI 4.1+, MPICH 4.0+.

6.4.2 6.4.2. Analyseurs Dynamiques de Mémoire (*Sanitizers* et Valgrind)

Pour interdire toute régression silencieuse : 1. **Compilation Instrumentée ASan / UBSan :** `bash cmake -B build_san -DCMAKE_BUILD_TYPE=Debug \ -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer" \ -DCMAKE_Fortran_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer"` Valide l'absence totale de dépassements de tampons (*buffer overflows*) sur les mailles fantômes. 2. **Audit Valgrind Memcheck :** `bash valgrind --leak-check=full --show-leak-kinds=all --error-exitcode=1 ./covof_tests` Garantit une allocation mémoire nette de zéro octet lors de la destruction des objets (`Simulation::~Simulation()`).

6.4.3 6.4.3. Test de Parité Stricte C++ $\leftrightarrow$ Fortran

Le script d'intégration lance simultanément le solveur C++ et le solveur Fortran sur la même configuration JSON et vérifie que la différence maximale relative sur chaque champ scalaire reste inférieure à la tolérance binaire :

$$\max_{i,j} \frac{|\phi_{\mathrm{C++}}(i,j) - \phi_{\mathrm{Fortran}}(i,j)|}{\max(|\phi_{\mathrm{C++}}|, 10^{-12})} < 10^{-12}$$

Ce test garantit que les deux codes demeurent des jumeaux numériques parfaits au cours de leur cycle de vie.

Chapitre 7

Standards de Codage, Profilage et Guide d'Extension

Pour pérenniser le co-développement gémellaire C++17 / Fortran 2018 et garantir une maintenabilité à long terme sur des décennies, COVOF 2D s'appuie sur des standards d'ingénierie logicielle rigoureux, une méthodologie d'analyse de performance fine et un protocole formel pour l'intégration de nouvelles fonctionnalités.

Ce chapitre formalise les règles de style de code, les outils de profilage HPC et guide le développeur pas à pas dans l'extension du solveur à travers un exemple pratique complet.

7.1 7.1. Standards de Codage et Conventions d'Ingénierie

7.1.1 7.1.1. Conventions C++17

Le code C++ respecte les directives du standard moderne C++17 et la norme *C++ Core Guidelines* de Bjarne Stroustrup et Herb Sutter :

1. **Formatage automatique :** Tout code doit être formaté via l'outil `clang-format` avec la configuration définie dans le fichier `.clang-format` racine (style basé sur Google C++ avec indentation à 4 espaces, limite de ligne à 100 colonnes).
2. **Gestion RAII stricte (*Resource Acquisition Is Initialization*) :**
   — Aucune gestion manuelle de mémoire via `new` ou `delete`.
   — Utilisation systématique de conteneurs standards (`std::vector`) ou de pointeurs intelligents à propriété exclusive (`std::unique_ptr`) et partagée (`std::shared_ptr`).
3. **Const-correctness et Noexcept :**
   — Toute méthode ne modifiant pas l'état interne de la classe doit être qualifiée `const`.
   — Les accesseurs triviaux et les destructeurs doivent être marqués `noexcept` pour assister le compilateur dans l'optimisation des sauts.
4. **Bannissement des macros pour les constantes :**
   — Remplacement de `#define PI 3.14159...` par `constexpr double PI = 3.14159265358979323846;`.
5. **Absence totale de variables globales mutables :**
   — Toutes les grandeurs d'état résident dans les instances explicites de `Simulation`, `Mesh2D` et `FlowFields`.

7.1.2 7.1.2. Conventions Fortran 2018

Le code Fortran adhère aux meilleures pratiques du calcul scientifique moderne :

1. **Directive `implicit none` obligatoire :**
   — Chaque module et chaque sous-programme doit débuter par `implicit none` sans exception.
2. **Encapsulation modulaire `private` par défaut :**
   — Tout module débute par la clause `private`, suivie de la liste explicite des symboles publics autorisés (`public :: ns_apply_system_matrix, ...`).
3. **Spécificateurs d'intention exhaustifs :**
   — Chaque argument formel de fonction ou sous-routine doit impérativement déclarer son intention :
   — `intent(in)` : Argument lu uniquement.
   — `intent(out)` : Argument initialisé et écrit par la routine.
   — `intent(inout)` : Argument lu et modifié.
4. **Interdiction des idiomes obsolètes :**
   — Bannissement formel des blocs `COMMON`, des instructions `EQUIVALENCE`, des `GOTO` calculés, et de l'attribut `SAVE` implicite.
5. **Types dérivés et procédures liées :**
   — Modélisation objet par types dérivés extensibles (`type, public :: mesh_2d_t`) avec interfaces génériques et surcharge d'opérateurs si pertinent.

7.2 7.2. Profilage HPC et Optimisation des Performances

Sur supercalculateur, l'optimisation ne procède jamais par intuition subjective mais par métrologie instrumentée rigoureuse.

7.2.1 7.2.1. Profilage Matériel sous Linux via `perf`

L'outil standard `perf` permet d'inspecter les compteurs de performance matérielle du processeur (*Hardware Performance Counters*) sans surcoût d'instrumentation :

```
# Mesure globale des compteurs matériels
perf stat -e cycles,instructions,cache-misses,L1-dcache-load-misses,LLC-load-misses \
     ./bin/covof_cpp params.json

# Analyse du nombre d'instructions par cycle (IPC)
# Un IPC < 1.0 indique un engorgement mémoire (Memory Stalls)
# Un IPC > 2.0 indique une excellente saturation des unités d'exécution vectorielles.
```

Pour localiser précisément les fonctions les plus consommatrices (*hot spots*) :

```
perf record -g --call-graph dwarf ./bin/covof_cpp params.json
perf report --stdio
```

7.2.2 7.2.2. Analyse Approfondie avec Intel VTune Profiler

Pour les architectures x86 Intel Xeon (Cascade Lake, Sapphire Rapids, Emerald Rapids), Intel VTune fournit une cartographie complète : 1. **Memory Access Analysis :** Mesure la bande passante consommée sur les canaux DDR5 / HBM et identifie les accès mémoire non séquentiels (*strided access*). 2. **Roofline Model :** Positionne les boucles de calcul critiques (diffusion visqueuse, projection de pression) sur le graphique intensité arithmétique (FLOPs/octet) versus performance crête (GFLOPs/s). Elle détermine immédiatement si l'algorithme est limité par la mémoire (*bandwidth bound*) ou par la puissance de calcul (*compute bound*).

7.2.3 7.2.3. Profilage des Communications MPI Distribuées avec Scalasca et Score-P

Pour traquer les goulets d'étranglement de scalabilité sur des centaines de cœurs :

```
# Compilation instrumentée Score-P
scorep mpicxx -O3 -std=c++17 src/*.cpp -o covof_scorep

# Exécution avec traçage OTF2
export SCOREP_EXPERIMENT_DIRECTORY=scorep_trace
mpirun -np 64 ./covof_scorep params.json

# Visualisation des déséquilibres avec Cube GUI
cube scorep_trace/profile.cubex
```

Ce profilage identifie immédiatement le temps d'attente parasite (*wait time*) induit par un déséquilibre de charge (*load imbalance*) dans les appels non bloquants `MPI_Waitall` des mailles fantômes.

7.3 7.3. Guide Développeur Pas-à-Pas : Ajout d'une Nouvelle Fonctionnalité

Pour illustrer de manière concrète et opérationnelle la procédure d'extension du solveur, nous détaillons ci-après l'implémentation complète de bout en bout de l'**équation de transport d'un scalaire passif** $S(\mathbf{x}, t)$ (représentant par exemple la concentration d'un traceur chimique ou d'un polluant) :

$$\frac{\partial S}{\partial t} + \nabla \cdot (\mathbf{u} S) = \nabla \cdot (D_s \nabla S)$$

7.3.1 Étape 1 : Ajout de la Clé de Configuration dans le Dictionnaire JSON

1. **Dans `include/covof/Config.hpp` :**

```
struct Config {
    // ... paramètres existants ...
    bool   enable_scalar_transport = false;
    double scalar_diffusivity      = 1.0e-5; // m^2/s
};
```

2. **Dans `src/Config.cpp` :**

```
if (root.contains("enable_scalar_transport")) {
    cfg.enable_scalar_transport = root["enable_scalar_transport"].get<bool>();
}
if (root.contains("scalar_diffusivity")) {
    cfg.scalar_diffusivity = root["scalar_diffusivity"].get<double>();
}
```

3. **Côté Fortran dans `COVOF_2D-FORTRAN/src/module_config.f90` :** Ajouter les champs dans le type dérivé `config_t` et leur lecture dans la routine d'analyse JSON.

7.3.2 Étape 2 : Déclaration et Allocation du Champ S

1. **Dans `include/covof/Field.hpp` :**

```
struct FlowFields {
    // ... champs existants ...
    std::vector<double> S;     // Scalaire passif aux centres de mailles
    std::vector<double> S_old; // Étage temporel n

    void allocate(int nxt, int nyt) {
        // ...
        S.assign(nxt * nyt, 0.0);
        S_old.assign(nxt * nyt, 0.0);
    }
};
```

2. **Dans `COVOF_2D-FORTRAN/src/module_flow_fields.f90` :** Ajouter `real(dp), allocatable :: S(:, :)` et allouer le tableau dans `fields_allocate_2d`.

7.3.3 Étape 3 : Implémentation du Noyau Numérique d'Advection-Diffusion

1. **Dans `include/covof/NavierStokesSolver.hpp` et `src/NavierStokesSolver.cpp` :** Ajouter la méthode `advancePassiveScalar` :

```
void NavierStokesSolver::advancePassiveScalar(FlowFields& f, double dt) {
    if (!config_.enable_scalar_transport) return;

    const int nxt = mesh_.nx_total();
    const double idx = 1.0 / mesh_.dx();
    const double idy = 1.0 / mesh_.dy();
    const double Ds  = config_.scalar_diffusivity;

    // Boucle de mise à jour explicite du scalaire
    #pragma omp parallel for collapse(2)
    for (int j = 1; j <= mesh_.ny_local(); ++j) {
        for (int i = 1; i <= mesh_.nx_local(); ++i) {
            // Flux diffusifs de Fick aux faces
            double diff_x = Ds * ((f.S[IDX(i+1, j)] - f.S[IDX(i, j)]) * idx -
                                  (f.S[IDX(i, j)] - f.S[IDX(i-1, j)]) * idx) * idx;
            double diff_y = Ds * ((f.S[IDX(i, j+1)] - f.S[IDX(i, j)]) * idy -
                                  (f.S[IDX(i, j)] - f.S[IDX(i, j-1)]) * idy) * idy;

            // Flux convectifs Upwind
            double u_e = f.u[IDX_U(i, j)];
            double u_w = f.u[IDX_U(i-1, j)];
            double v_n = f.v[IDX_V(i, j)];
            double v_s = f.v[IDX_V(i, j-1)];

            double flux_e = (u_e > 0.0) ? u_e * f.S[IDX(i, j)] : u_e * f.S[IDX(i+1, j)];
            double flux_w = (u_w > 0.0) ? u_w * f.S[IDX(i-1, j)] : u_w * f.S[IDX(i, j)];
            double flux_n = (v_n > 0.0) ? v_n * f.S[IDX(i, j)] : v_n * f.S[IDX(i, j+1)];
            double flux_s = (v_s > 0.0) ? v_s * f.S[IDX(i, j-1)] : v_s * f.S[IDX(i, j)];

            double conv = (flux_e - flux_w) * idx + (flux_n - flux_s) * idy;

            // Avancement Euler
            f.S[IDX(i, j)] = f.S_old[IDX(i, j)] + dt * (diff_x + diff_y - conv);
        }
    }
}
```

2. **Équivalent Fortran 2018 dans `module_navier_stokes.f90` :** Créer la sous-routine `subroutine ns_advance_passive_scalar(fields, mesh, cfg, dt)`.

7.3.4 Étape 4 : Mise à Jour des Mailles Fantômes MPI

Dans la méthode `Simulation::timeStep()`, intercaler la synchronisation MPI du nouveau champ :

```
mpi_comm_.exchangeHalos(fields_.S, mesh_.nx_total(), mesh_.ny_total(), mesh_.nghost());
```

7.3.5 Étape 5 : Exportation dans les Fichiers VTK

Dans `src/VTKWriter.cpp`, ajouter la section de données scalaires `S` dans l'arbre XML `<CellData>` :

```
if (config.enable_scalar_transport) {
    writeScalarField(vtr_file, "passive_scalar", fields.S);
}
```

7.3.6 Étape 6 : Validation par Solution Analytique MMS

Créer un cas de test unitaire vérifiant la décroissance exponentielle d'un mode de Fourier sinusoïdal en diffusion pure ($u = v = 0$) :

$$S_{\text{anal}}(x, y, t) = S_0 \sin\left(\frac{2\pi x}{L}\right) \sin\left(\frac{2\pi y}{H}\right) e^{-D_s\left(\frac{4\pi^2}{L^2}+\frac{4\pi^2}{H^2}\right)t}$$

Vérifier que l'erreur relative entre la solution numérique et la solution analytique décroît au taux théorique attendu de $O(\Delta x^2 + \Delta t)$.